HBase基本原理与程序设计



摘要

- HBase基本工作原理
- HBase基本操作
- HBase编程方法示例



摘要

- HBase基本工作原理
- HBase基本操作
- HBase编程方法示例



HBase

- Hadoop可以很好地解决大规模数据的离线批量处理问题，但是，受限于Hadoop MapReduce编程框架的高延迟数据处理机制，使得Hadoop无法满足大规模数据实时处理应用的需求。
- HDFS面向批量访问模式，不是随机访问模式。
- 传统的通用关系型数据库无法应对在数据规模剧增时导致的系统扩展性和性能问题（分库分表也不能很好解决）。
- 传统关系数据库在数据结构变化时一般需要停机维护；空列浪费存储空间。
- 因此，业界出现了一类面向半结构化数据存储和处理的高可扩展、低写入/查询延迟的系统，例如，键值数据库、文档数据库和列族数据库（如BigTable和HBase等）。



HBase

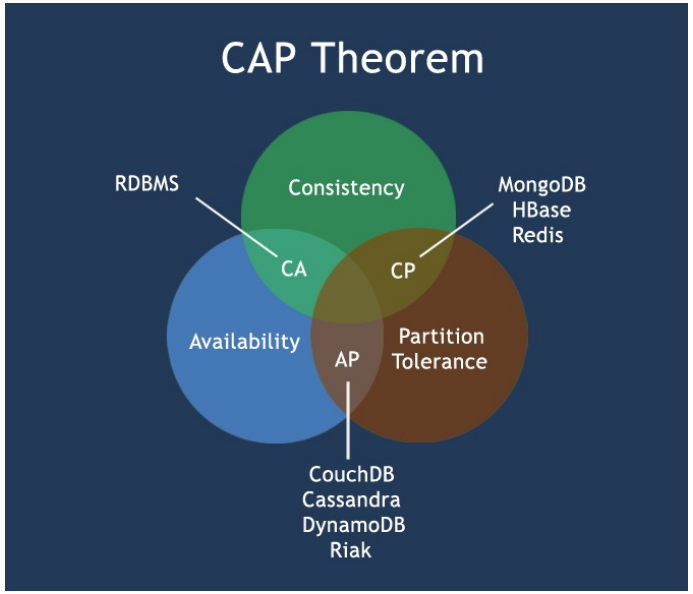
- HBase是一个高可靠、高性能、面向列、可伸缩的分布式数据库，是谷歌BigTable的开源实现，主要用来存储非结构化和半结构化的松散数据。HBase的目标是处理非常庞大的表，可以通过水平扩展的方式，利用廉价计算机集群处理由超过10亿行数据和数百万列元素组成的数据表。

HBase的设计目标

- 针对HDFS缺少结构化半结构化数据存储访问能力的缺陷，提供一个分布式数据管理系统，解决大规模的结构化和半结构化数据存储访问问题
- Google BigTable的一个开源实现
- 提供基于列存储模式的大数据表管理能力
- 可存储管理数十亿以上的数据记录，每个记录可包含百万以上的数据列
- HBase试图提供随机和实时的数据读写访问能力
- 具有高可扩展性、高可用性、容错处理能力、负载平衡能力、实时数据查询能力

HBase的功能特点

- 列式存储
- 表数据是稀疏的多维映射表
- 读写的严格一致性（区别于Cassandra的最终一致性）
- 提供很高的数据读写速度，为写数据进行了特别优化
- 良好的线形可扩展性
- 提供海量数据存储能力
- 数据会自动分片
- 具有自动的失效检测和恢复能力，保证数据不丢失
- 提供了方便的与HDFS和MapReduce集成的能力
- 提供Java API作为编程接口





HBase vs. RDBMS

- 数据类型：RDBMS采用关系模型，具有丰富的数据类型和存储方式，HBase则采用了更加简单的数据模型，它把数据存储为未经解释的字符串。
- 数据操作： RDBMS中包含了丰富的操作，其中会涉及复杂的多表连接。HBase操作则不存在复杂的表与表之间的关系，只有简单的插入、查询、删除、清空等，因为HBase在设计上就避免了复杂的表和表之间的关系。
- 存储模式： RDBMS是基于行模式存储的。HBase是基于列存储的，每个列族都由几个文件保存，不同列族的文件是分离的。



HBase vs. RDBMS

- 数据索引： RDBMS通常可以针对不同列构建复杂的多个索引，以提高数据访问性能。HBase只有一个索引——行键，通过巧妙的设计，HBase中的所有访问方法，或者通过行键访问，或者通过行键扫描，从而使得整个系统不会慢下来。
- 数据维护：在RDBMS中，更新操作会用最新的当前值去替换记录中原来的旧值，旧值被覆盖后就不会存在。而在HBase中执行更新操作时，并不会删除数据旧的版本，而是生成一个新的版本，旧有的版本仍然保留。
- 可伸缩性： RDBMS很难实现横向扩展，纵向扩展的空间也比较有限。相反，HBase和BigTable这些分布式数据库就是为了实现灵活的水平扩展而开发的，能够轻易地通过在集群中增加或者减少硬件数量来实现性能的伸缩。

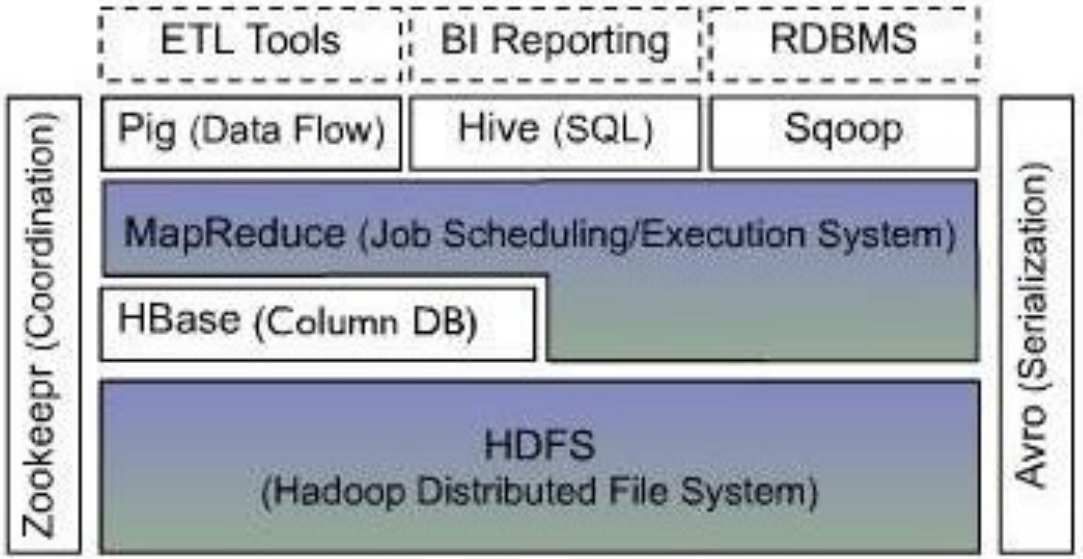


HBase访问接口

类型	特点	场合
Native Java API	最常规和高效的访问方式	适合Hadoop MapReduce作业并行批处理HBase表数据
HBase Shell	HBase的命令行工具，最简单的接口	适合HBase管理使用
Thrift Gateway	利用Thrift序列化技术，支持C++、PHP、Python等多种语言	适合其他异构系统在线访问HBase表数据
REST Gateway	解除了语言限制	支持REST风格的Http API访问HBase
Pig	使用Pig Latin流式编程语言来处理HBase中的数据	适合做数据统计
Hive	简单	当需要以类似SQL语言方式来访问HBase的时候

HBase在Hadoop中的生态环境

- 构建于分布式文件系统HDFS之上
- 为上层应用提供结构化半结构化海量数据存储访问能力



HBase的运行依赖于Hadoop HDFS文件系统提供数据的持久化（支持append功能），依赖Zookeeper提供集群的同步和协调。

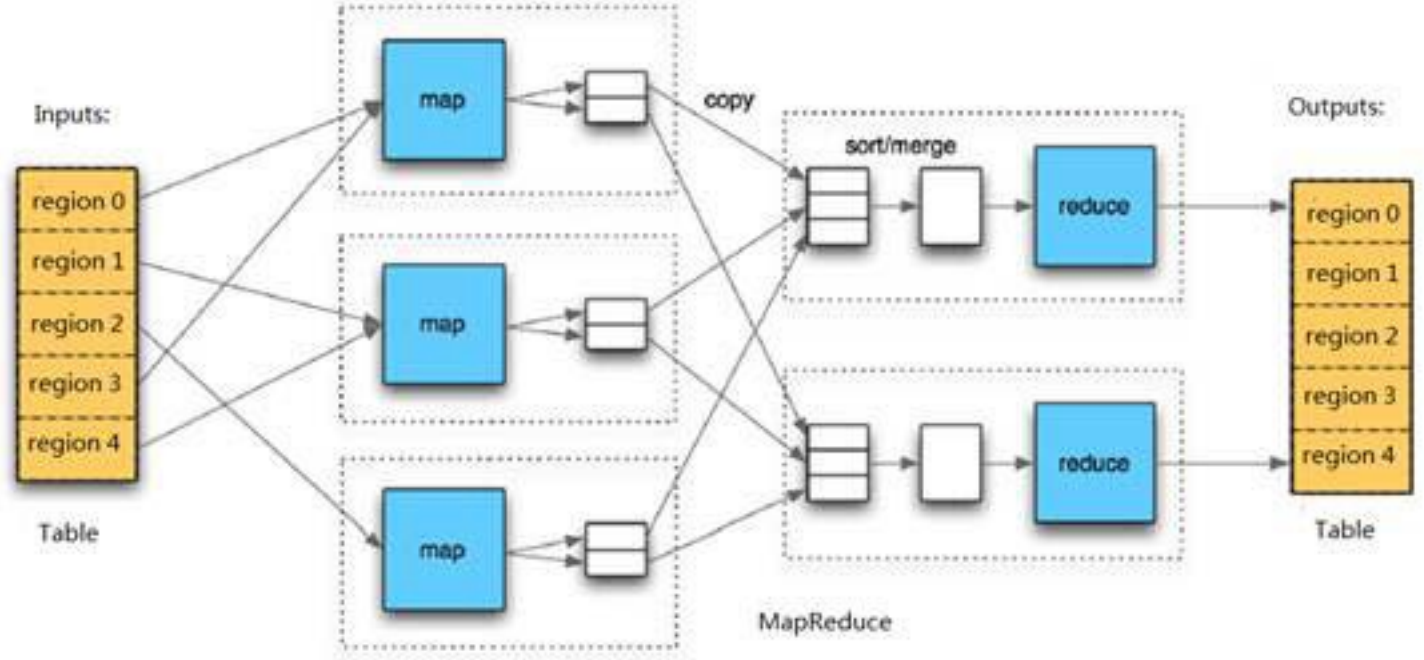


HBase在Hadoop中的生态环境

- 分布式协调服务器Zookeeper
  - 保证任何时候，集群中只有一个HBase Master
  - 实时监控Region Server的状态，将Region Server的上线和下线信息实时通知给HBase Master
  - 存储HBase目录表的寻址入口
  - 存储HBase的schema，包括有哪些表，每个表有哪些列族等各种元信息

HBase在Hadoop中的生态环境

- 可与MapReduce协同工作，为MapReduce提供数据输入输出，以完成数据的并行化处理



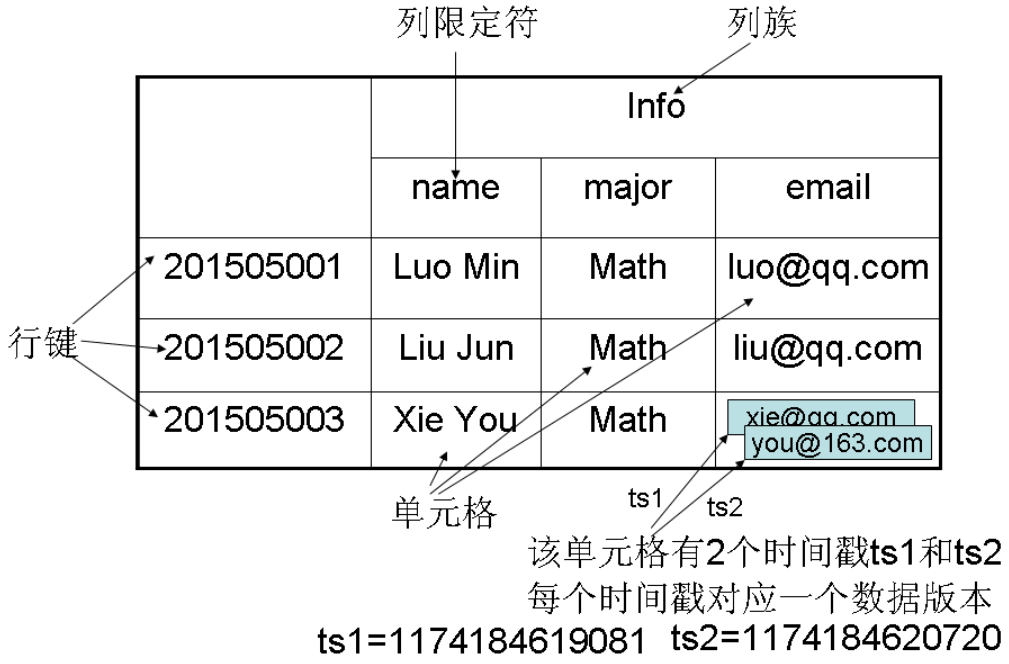


HBase数据模型

- HBase的数据模型是一个稀疏、多维度、排序的映射表，这张表的索引是行键、列族、列限定符和时间戳。
- 每个值是一个未经解释的字符串，没有数据类型。
- 用户在表中存储数据，每一行都有一个可排序的行键和任意多的列。
- 表在水平方向由一个或者多个列族组成，一个列族中可以包含任意多个列，同一个列族里面的数据存储在一起。
- 列族支持动态扩展，可以很轻松地添加一个列族或列，无需预先定义列的数量以及类型，所有列均以字符串形式存储，用户需要自行进行数据类型转换。
- HBase中执行更新操作时，并不会删除数据旧的版本，而是生成一个新的版本，旧有的版本仍然保留（这是和HDFS只允许追加不允许修改的特性相关的）。

HBase数据模型

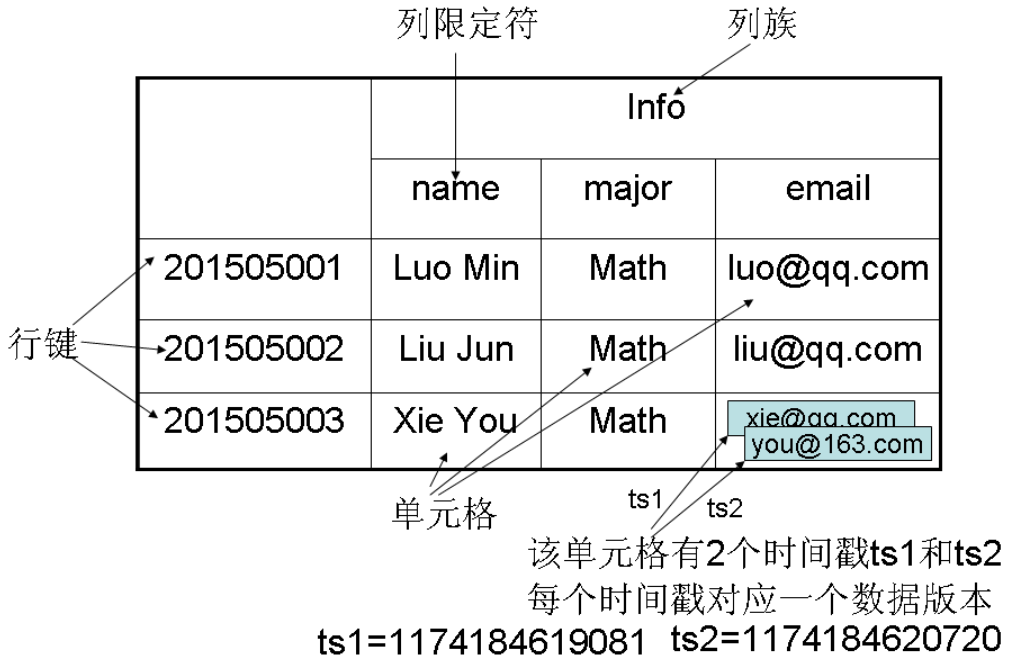
- 表：HBase采用表来组织数据，表由行和列组成，列划分为若干个列族
- 行：每个HBase表都由若干行组成，每个行由行键（row key）来标识。
- 列族：一个HBase表被分组成许多“列族”（Column Family）的集合，它是基本的访问控制单元
- 列限定符：列族里的数据通过列限定符（或列）来定位
- 单元格：在HBase表中，通过行、列族和列限定符确定一个“单元格”（cell），单元格中存储的数据没有数据类型，总被视为字节数组byte[]
- 时间戳：每个单元格都保存着同一份数据的多个版本，这些版本采用时间戳进行索引



数据坐标

- HBase中需要根据行键、列族、列限定符和时间戳来确定一个单元格，因此，可以视为一个“四维坐标”，即[行键, 列族, 列限定符, 时间戳]

键	值
[“201505003”, “Info”, “email”, 1174184619081]	“xie@qq.com”
[“201505003”, “Info”, “email”, 1174184620720]	“you@163.com”



概念视图

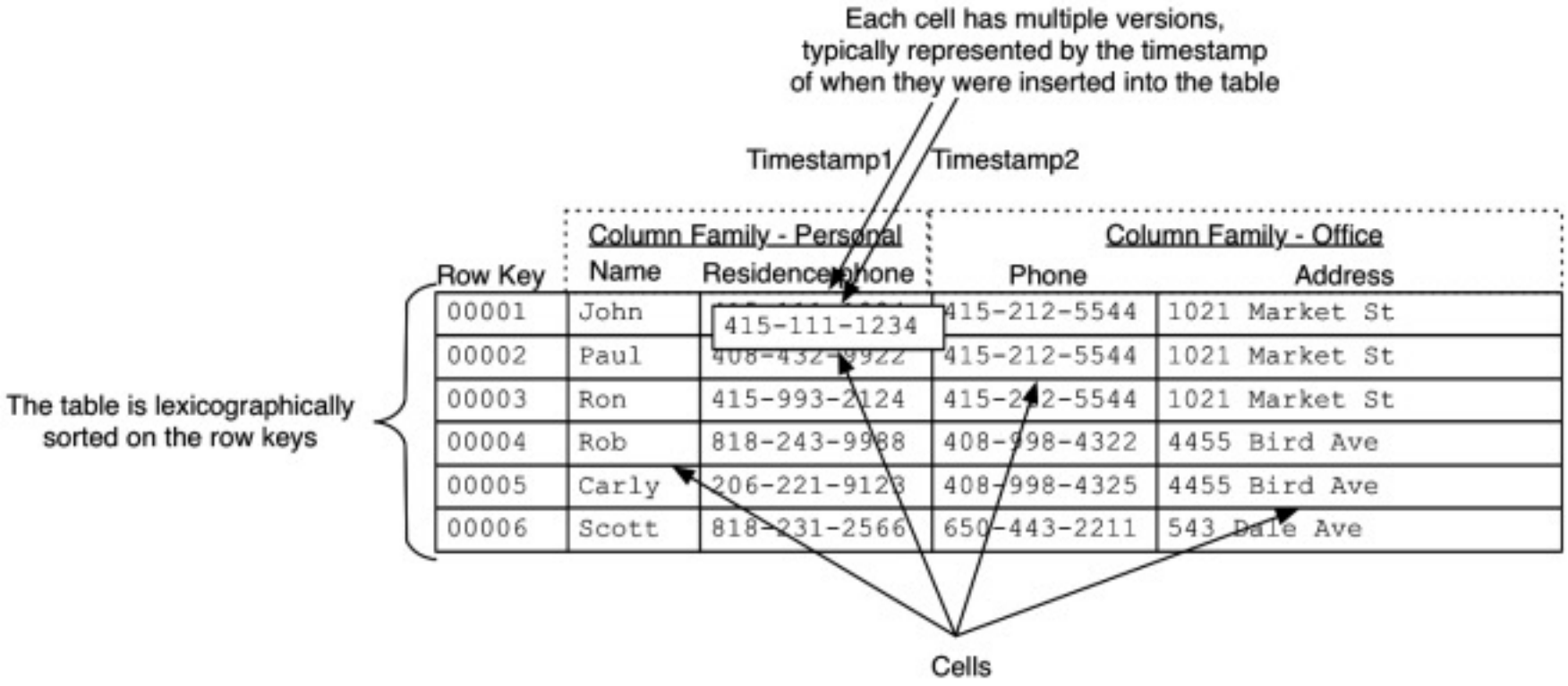


Figure 1: A table in HBase consisting of two column families, Personal and Office, each having two columns. The entity that contains the data is called a cell. The rows are sorted based on the row keys.

概念视图

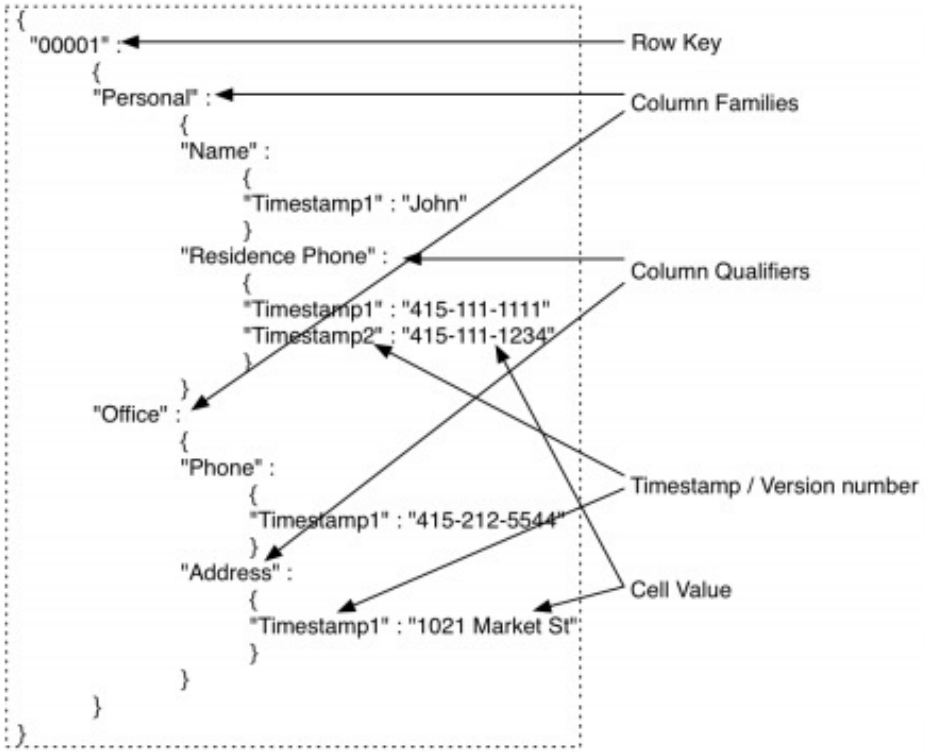


Figure 2: One row in an HBase table represented as a multidimensional map

HBase一定程度上又可以看成一个多维度的Map模型去理解它的数据模型。即以行键(Row Key)，列标识(column qualifier)，时间戳(timestamp)标识的有序Map数据结构的数据库，具有稀疏，分布式，持久化，多维度等特点。

一个行键映射一个列族数组，列族数组中的每个列族又映射一个列标识数组，列标识数组中的每一个列标识又映射到一个时间戳数组，里面是不同时间戳映射下不同版本的值，但是默认取最近时间的值，所以可以看成是列标识和它所对应的值的映射。

概念视图

00001 → { Personal : { Name : { Timestamp1 : John }, Residence Phone : { Timestamp1 : 415-111-1234 } },
{ Office : { Phone : { Timestamp1 : 415-212-5544 }, Address : { Timestamp1 : 1021 Market St } } }

00001 , Personal → { Name : { Timestamp1 : John }, Residence Phone : { Timestamp1 : 415-111-1234 } }

00001, Personal:Residence Phone → { {Timestamp1 : 415-111-1111 } , { Timestamp2 : 415-111-1234) }

00001, Personal:Residence Phone , Timestamp1 → { 415-111-1111 }

00001, Personal:Residence Phone , Timestamp2 → { 415-111-1234 }

Figure 3: HBase table as a key-value store. The key can be considered to be just the row key or a combination of the row key, column family, qualifier, timestamp, depending on the cells that you are interested in addressing. If all the cells in a row were of interest, the key would be just the row key. If only specific cells are of interest, the appropriate column families and qualifiers will need to be a part of the key

HBase一定程度上也可以看成是一个类似Redis那样的Key-Value数据库。当你要查询某一行的所有数据时，Row Key就相当于Key，而Value就是单元中的数据



行主键

- 行主键row key是用来检索记录的主键。这样的话，访问HBase table中的行，有三种方式：1 通过单个row key访问；2 通过row key的范围range来访问；3 全表扫描。
- 行键（Row key）可以是任意字符串（最大长度是64KB，实际应用中长度一般为10-100bytes），在HBase内部，row key保存为字节数组。
- 存储时，数据按照Row key的字典序（byte order）排序存储。这样的话，设计key时，要充分利用排序存储这个特性，将经常一起读取的行存储放到一起。（空间局部性）
- 行的一次读写是原子操作（不论一次读写多少列）。这样的设计兼顾了用户可以理解在一行中的读写行为以及设计上的可扩展性。



列族

- 列族的设计与传统数据库中的列不一致
- 与BigTable中的模式一样，HBase表中的每个列，都归属于某个列族。列族是表的schema的一部分，必须在使用表之前定义。列名都以列族作为前缀。例如courses:history，courses:math都属于courses这个列族。
- 访问控制、磁盘和内存的使用统计都是在列族层面进行的。实际应用中，列族上的控制权限能帮助管理不同类型的应用：允许一些应用可以添加新的基本数据、一些应用可以读取基本数据并创建继承的列族、一些应用则只允许浏览数据（甚至可能因为隐私的原因不能浏览所有数据）。
  - 在具体实现上，一张表中的不同列族是分开独立存放的。就是说，如果有两个列族family1和family2，那么在HDFS存储时，family1是一组文件，family2是另外一组文件，两者绝不混合存储。



时间戳timestamp

- HBase中通过row和column确定的一个存贮单元称为单元格cell。每个cell都保存着同一份数据的多个版本。版本通过时间戳来索引。时间戳的类型是64位整型。时间戳可以由HBase（在数据写入时自动）赋值，此时时间戳是精确到毫秒的当前系统时间。时间戳也可以由客户显式赋值。如果应用程序要避免数据版本冲突，就必须自己生成具有唯一性的时间戳。每个cell中，不同版本的数据按照时间倒序排序，即最新的数据排在最前面。
- 为了避免数据存在过多版本造成的管理（包括存贮和索引）负担，HBase提供了两种数据版本回收方式。一是保存数据的最后n个版本，二是保存最近一段时间内的版本（比如最近七天）。用户可以针对每个列族进行设置。

物理视图

行关键字	时间戳	列"contents:"	
"com.cnn.www"	t6	"<html>…"	
	t5	"<html>…"	
	t3	"<html>…"	
行关键字	时间戳	列 "anchor:"	
"com.cnn.www"	t9	"anchor:cnnsi.com"	"CNN"
	t8	"anchor:my.look.ca"	"CNN.com"
行关键字	时间戳	列 "mime:"	
"com.cnn.www"	t6	"text/html"	

- 按照列存储的稀疏行/列矩阵。物理存储格式上按逻辑模型中的行进行分割，并按照列族存储。
- 值为空的列不予存储。



物理视图

列族contents

行键	时间戳	列族contents
"com.cnn.www"	t6	contents:html="<html>..."
	t5	contents:html="<html>..."
	t3	contents:html="<html>..."

列族anchor

行键	时间戳	列族anchor
"com.cnn.www"	t9	anchor:cnnsi.com=”CNN”
	t8	anchor:my.look.ca="CNN.com"

面向列的存储 vs 面向行的存储

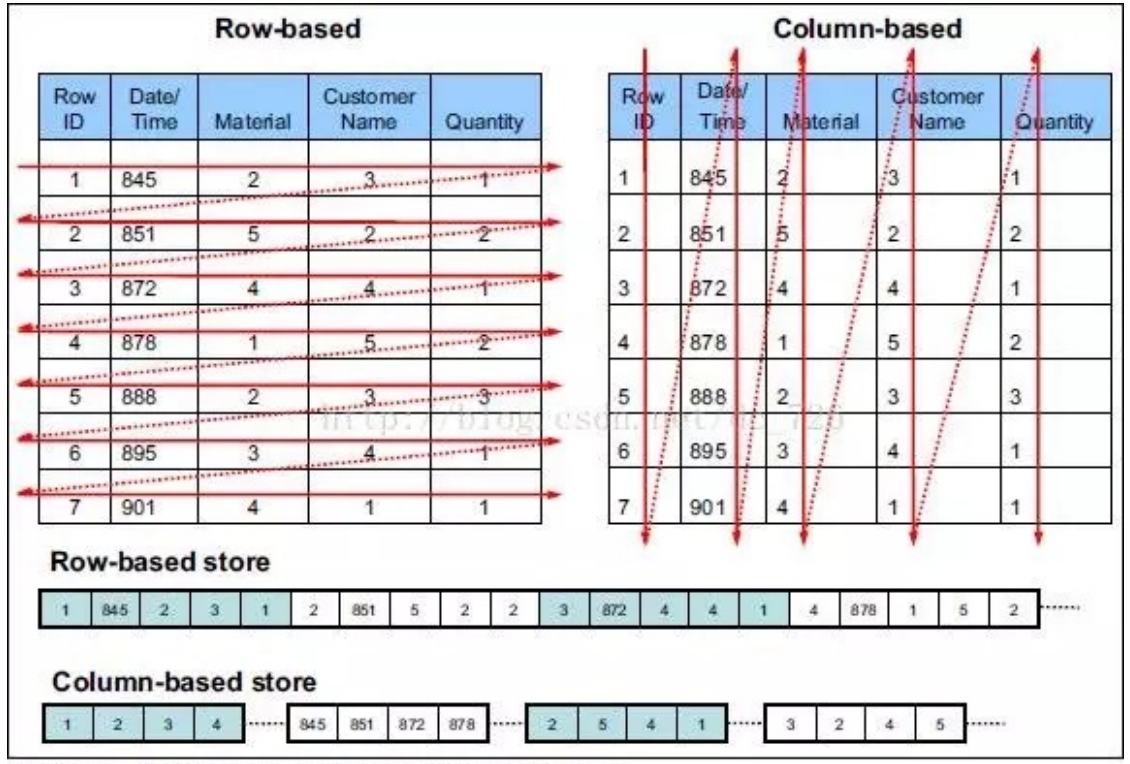
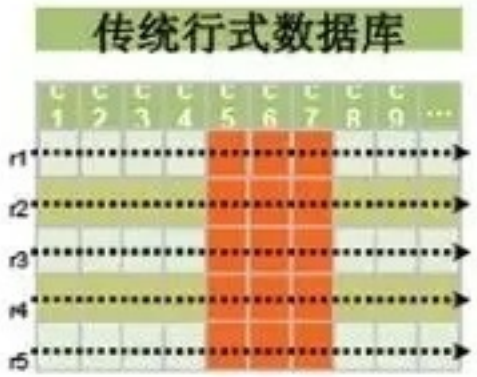


Figure 1-4 Row-based and column-based storage models

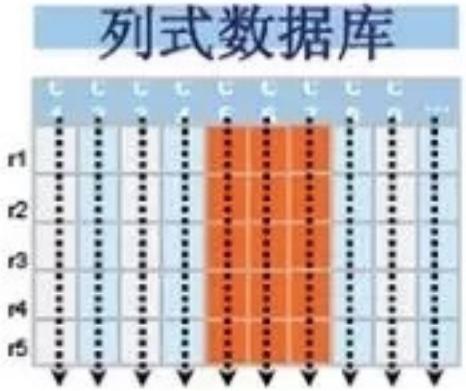
行式数据库和列式数据库示意图

面向列的存储 vs 面向行的存储



- 数据是按行存储的
- 没有索引的查询使用大量I/O
- 建立索引和物化视图需要花费大量时间和资源
- 面对查询的需求，数据库必须被大量膨胀才能满足性能要求

行式更适合OLTP，比如传统的基于增删改查操作的应用



- 数据按列存储 - 每一列单独存放
- 数据即是索引
- 只访问查询涉及的列 - 大量降低系统IO
- 每一列由一个线索来处理 - 查询的并发处理
- 数据类型一致，数据特征相似 - 高效压缩

列式更适合OLAP，非常适合于数据仓库领域，比如数据分析、海量存储和商业智能；涉及不经常更新的数据

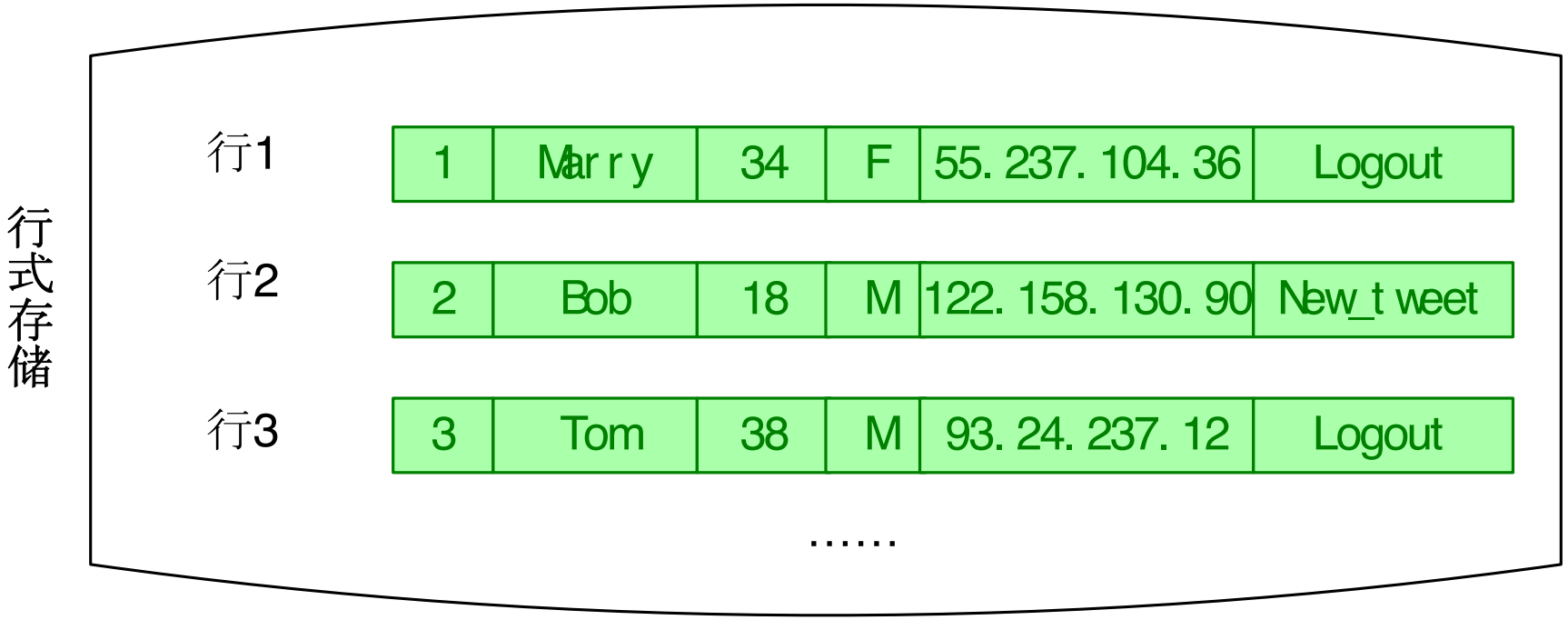
行式数据库和列式数据库对比

面向列的存储 vs 面向行的存储

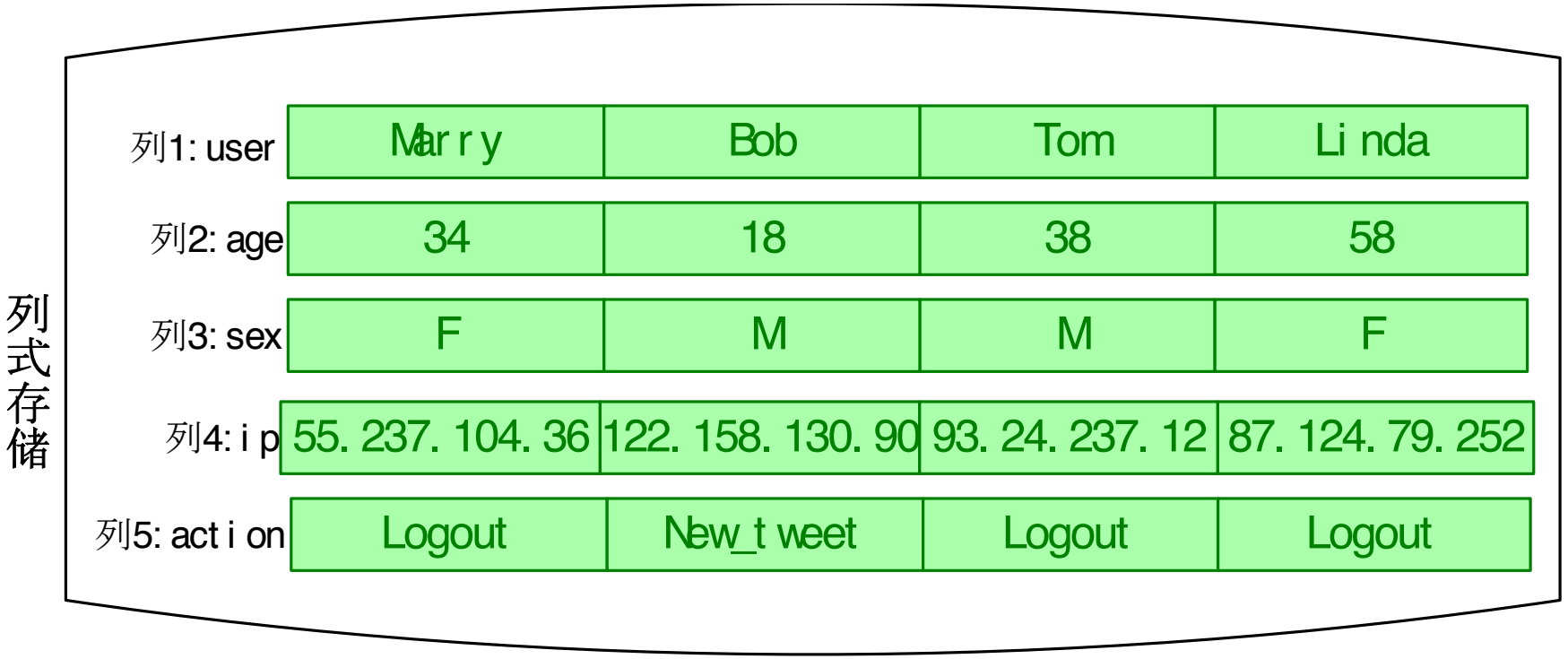
SQL模式

Log					
Log_id	user	age	sex	ip	action
1	Marry	34	F	55.237.104.36	Logout
2	Bob	18	M	122.158.130.90	New_tweet
3	Tom	38	M	93.24.237.12	Logout
4	Linda	58	F	87.124.79.252	Logout

面向列的存储 vs 面向行的存储



面向列的存储 vs 面向行的存储



面向列的存储 vs 面向行的存储

列式数据库——大幅降低磁盘I/O

- 范例：计算1天中某支股票的平均价格



列式数据库在并行查询处理和压缩上更有优势。而且数据是以列为单元存储，完全不用考虑数据建模或者说建模更简单了。要查询计算哪些列上的数据，直接读取列就行。



HBase的查询模式

- HBase通过行键、列族、列限定符和时间戳来确定一个存储单元，即：
  - {row key, column family: column name, timestamp} → value
- HBase可以支持的查询方式：
  - 通过单个row key访问
  - 通过row key的范围来访问
  - 全表扫描
- 合理设计row key



HBase表设计

- 表的规范化设计
  - 传统数据库：通常需要满足第三范式（3NF）
    - 第三范式：指表中的所有数据元素不但要能唯一地被主关键字所标识，而且它们之间还必须相互独立，不存在其他的函数关系。
  - NoSQL数据库：反规范化。应该将相关的数据都存放在一起，不担心冗余。



HBase表设计

- Row key的设计要点
  - 例1：收集一个集群（4000个节点）中的所有log并在HBase表LOG_DATA中存储。需要收集的字段有：（机器名，时间，事件，事件正文）
  - 插入操作：尽可能高效地插入
  - 查询操作：
    - 需求一：对于某台机器，查询一个大的时间段（例如1个月）内的所有满足条件的记录（单机查询）
    - 需求二：查询某个时间段内对所有机器满足条件的记录（全局查询）



HBase表设计

- row key设计
  - 方案一：[机器名][时间][事件]
  - 方案二：[时间][机器名][事件]
  - 方案一对单机查询友好，对插入友好（Region分散，并发度高）
  - 方案二对全局查询友好，对插入不友好（Region集中，并发度低）
- row key设计的弥补措施
  - salted（加盐）：对单调数据，通过计算盐值，放在单调数据前让其不单调。例如：
    - 盐值 = 时间 % 桶个数
    - row key：[盐值][时间][机器名][事件]
    - 副作用：查询变慢，降低了系统的吞吐量，桶个数不宜太大



HBase表设计

- 表设计的选择
  - 一张大表 or 按时间进行分表？—— 根据应用需求选择
  - 大表设计：
    - 优点：查询都在单张表完成
    - 缺点：数据过期时需要依赖major compaction进行压缩，会造成大量的数据读写；活跃Region在RegionServer间的分布可能不均匀。
  - 按时间分表设计：
    - 优点：数据过期可以通过简单删除整张表完成；活跃Region可以得到分布均匀。
    - 缺点：需要跨表查询时，性能会比较差。



HBase表设计

- Row key的设计要点
  - 例2：设计一张表，用来保存微博上用户互粉的信息。
  - 读场景业务要求
    1. 每个用户都关注了谁
    2. 用户A有没有关注用户B
    3. 谁关注了用户A
  - 写场景业务要求
    1. 用户关注了另一个用户
    2. 用户取消关注某个用户

HBase表设计

- 初始设计

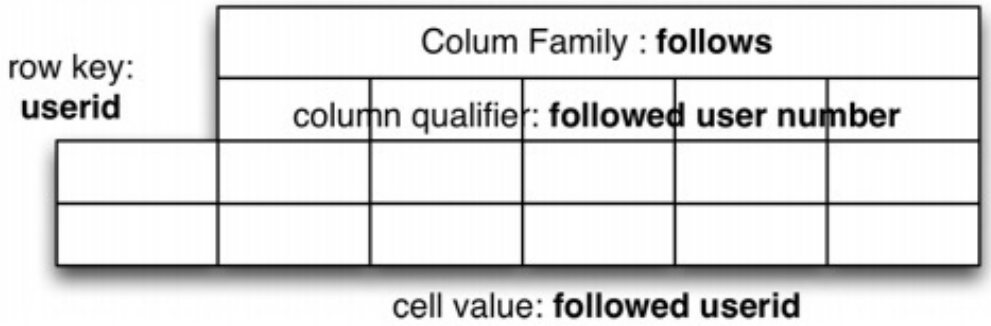


Figure 4: HBase table to persist the list of users a particular user is following

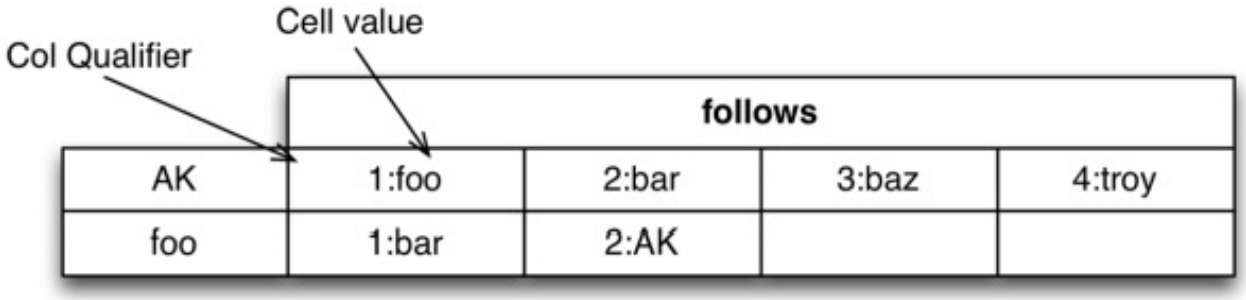


Figure 5: A table with sample data for the design shown in Figure 4

HBase表设计

- 改进列族序号方案

	follows				
AK	1:foo	2:bar	3:baz	4:troy	count:4
foo	1:bar	2:AK	count:2		

Figure 6: A table with sample data for the design shown in Figure 4 but with a counter to keep count of the number of users being followed by a given user

HBase表设计

- 改进列族序号方案

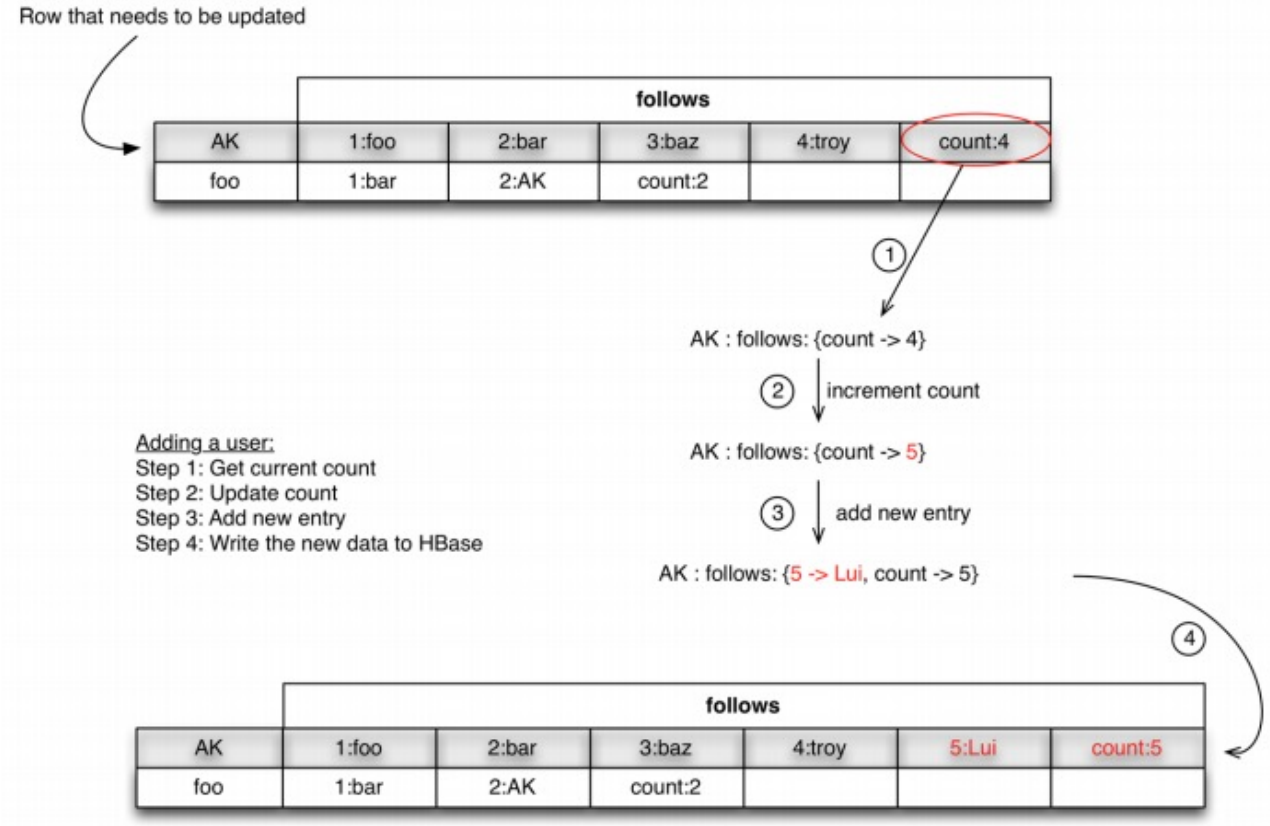


Figure 7: Steps required to add a new user to the list of followed users based on the table design from Figure 6

HBase表设计

- 改进列名方案

	follows			
AK	foo:1	bar:1	baz:1	troy:1
foo	bar:1	AK:1		

Figure 8: The relationship table with the cells now having the followed user's username as the column qualifier and an arbitrary string as the cell value.

HBase表设计

- 综合优化方案

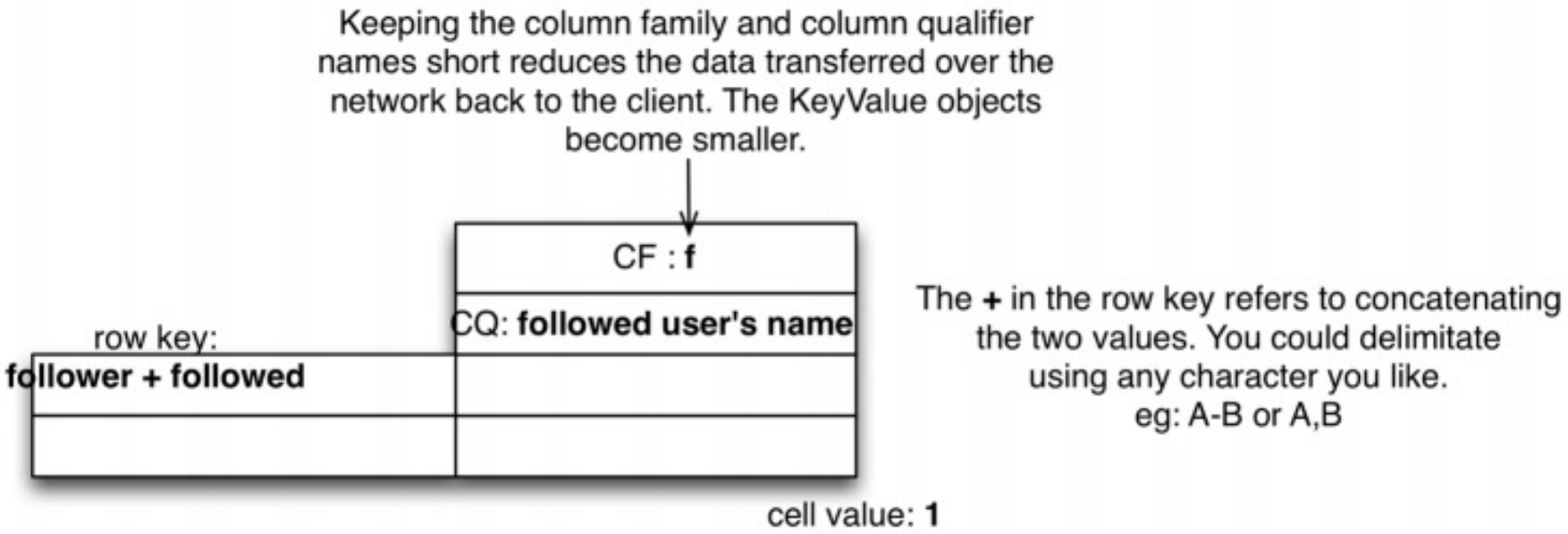


Figure 9: The relationship table with the row key containing the follower and the followed user

HBase表设计

- 综合优化方案

	f
AK+foo	James Foo:1
AK+bar	Jimmy Bar:1
AK+baz	Ricky Baz:1
AK+troy	Troy:1
foo+bar	Jimmy Bar:1
foo+AK	AK:1

Putting the user name in the column qualifier saves you from looking up the users table for the name of the user given an id. You can simply list out names or ids while looking at relationships just from this table. The downside of this is that you need to update the name in all the cells if the user updates their name in their profile.
This is classic *Denormalization*.

Figure 10: Relationship table based on the design shown in Figure 9 with some sample data

HBase表设计

- Row Key加密

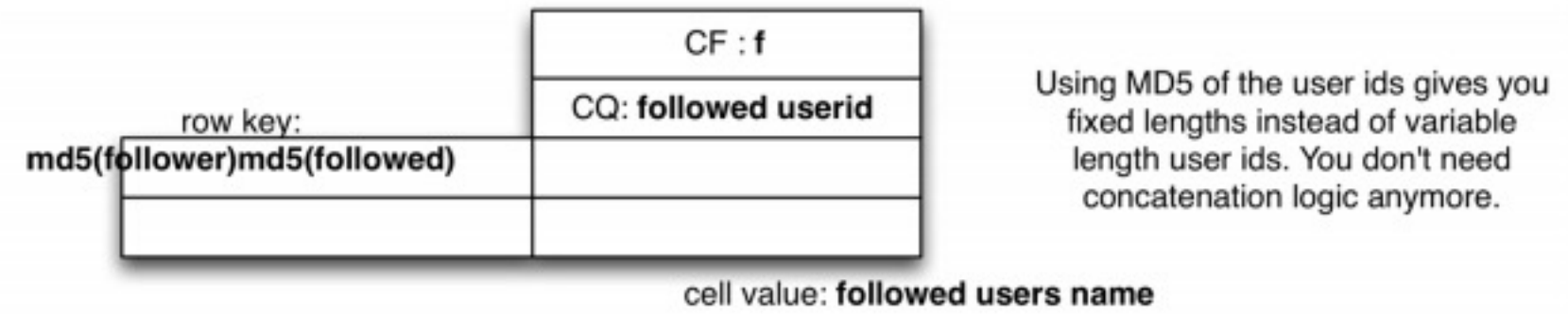


Figure 11: Using MD5s as a part of row keys to achieve fixed lengths. This also allows you to get rid of the + delimiter that we needed so far. The row keys now consist of fixed length portions, with each user ID being 16 bytes.



HBase表设计

- Row Key是HBase表结构设计中很重要的一环，它设计的好坏直接影响程序和HBase交互的效率和数据存储的性能。
- HBase的表结构比传统关系型数据库更灵活，能存储任何二进制数据在表中，而且无关数据类型。
- 在相同的列族中所有数据都具有相同的操作模式。
- 主要是通过Row Key来建立索引。



HBase表设计

- 以纵向扩张为主设计的表结构能快速简单的获取数据，但牺牲了一定的原子性，就比如上文中最后一种表结构；而以横向扩张为主设计的表结构，也就是列族中有很多列，比如上文中第一种表结构，能在行里面保持一定的原子性。
- HBase没有跨行事务，所有尽量在一次API请求操作中获取到结果。
- 对Row Key的Hash优化能获得固定长度的Row Key并使数据分布更加均匀一些，而不是集中在一台服务器上，但是也牺牲了一定的数据排序和读取性能。
- 可以利用列标识来存储数据。
- 列标识名字的长度和列族名字的长度都会影响I/O的读写性能和发送给客户端的数据量，所以它们的命名应该简洁。

HBase表设计的经验法则

- 把 region 的大小限制在 10 到 50 GB 之间。
- 限制 cell 的大小在 10 MB 之内，如果使用的是 MOB(Medium Object Storage)类型，限制在 50 MB 之内。否则，考虑把 cell 的数据存储在 HDFS 中，并在 HBase 中存储指向该数据的指针。
- 典型的 schema 每张表包含 1 到 3 个列族。HBase 表设计不应当和 RDBMS 表设计类似。
- 对于拥有 1 或 2 个列族的表来说，50-100 个 region 是比较合适的。请记住，region 是列族的连续段。

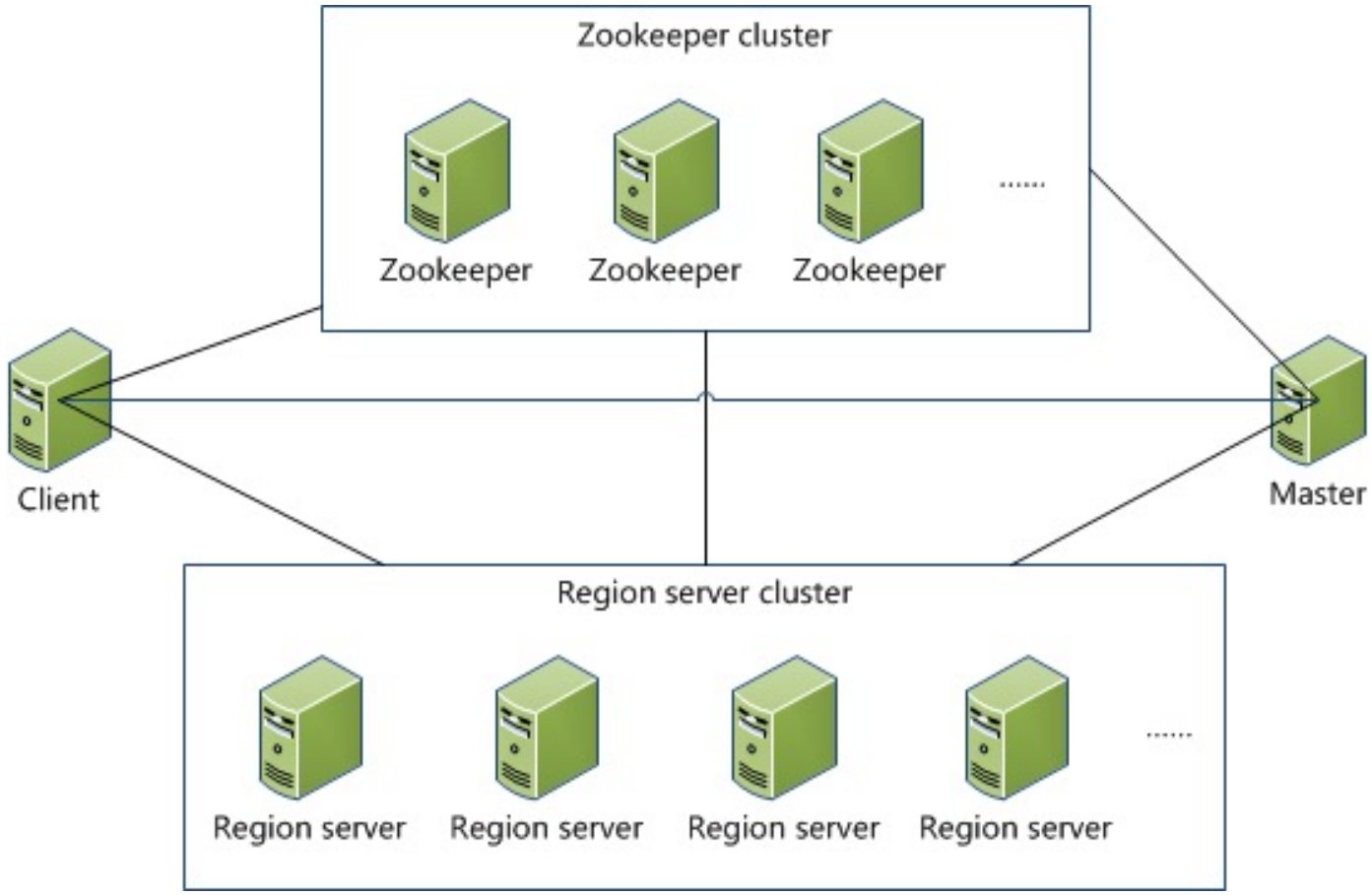


HBase表设计的经验法则

- 保持列族名称尽可能短。每个值都会存储列族的名称(忽略前缀编码)。它们不应该像典型 RDBMS 那样，是自文档化，描述性的名称。
- 如果你正在存储基于时间的机器数据或者日志信息，并且 row key 是基于设备 ID 或者服务 ID + 时间，最终会出现这样一种情况，即更旧的数据 region 永远不会有额外写入。在这种情况下，最终会存在少量的活动 region 和大量不会再有新写入的 region。对于这种情况，可以接受更多的 region 数量，因为资源的消耗只取决于活动 region。
- 如果只有一个列族会频繁写，那么只让这个列族占用内存。当分配资源的时候注意写入模式。



HBase的运行组成


Zookeeper cluster
Zookeeper
Zookeeper
Zookeeper
.....
Client
Master
Region server cluster
Region server
Region server
Region server
Region server
.....



HBase的运行组成

- 客户端
  - 客户端包含访问HBase的接口，同时在缓存中维护着已经访问过的Region位置信息，用来加快后续数据访问过程
- Zookeeper服务器
  - Zookeeper可以帮助选举出一个Master作为集群的总管，并保证在任何时刻总有唯一一个Master在运行，这就避免了Master的“单点失效”问题

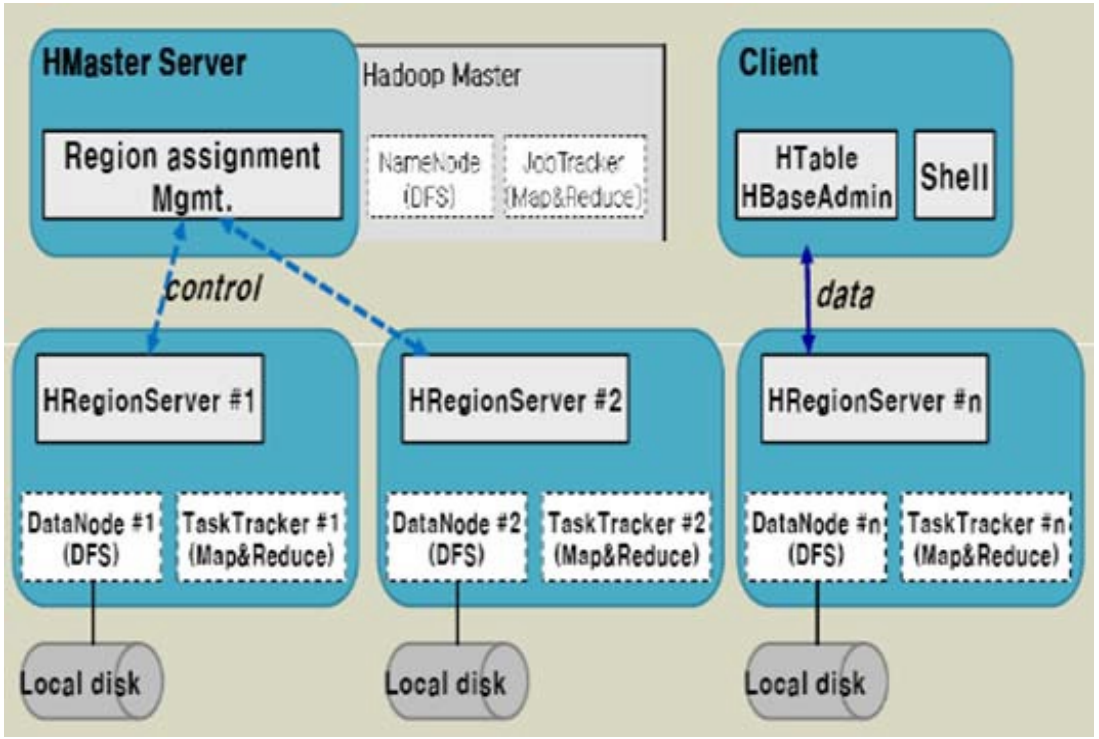


HBase的运行组成

- Master
  - 主服务器Master主要负责表和Region的管理工作：
    - 管理用户对表的增加、删除、修改、查询等操作
    - 实现不同Region服务器之间的负载均衡
    - 在Region分裂或合并后，负责重新调整Region的分布
    - 对发生故障失效的Region服务器上的Region进行迁移
- Region服务器
  - Region服务器是HBase中最核心的模块，负责维护分配给自己的Region，并响应用户的读写请求

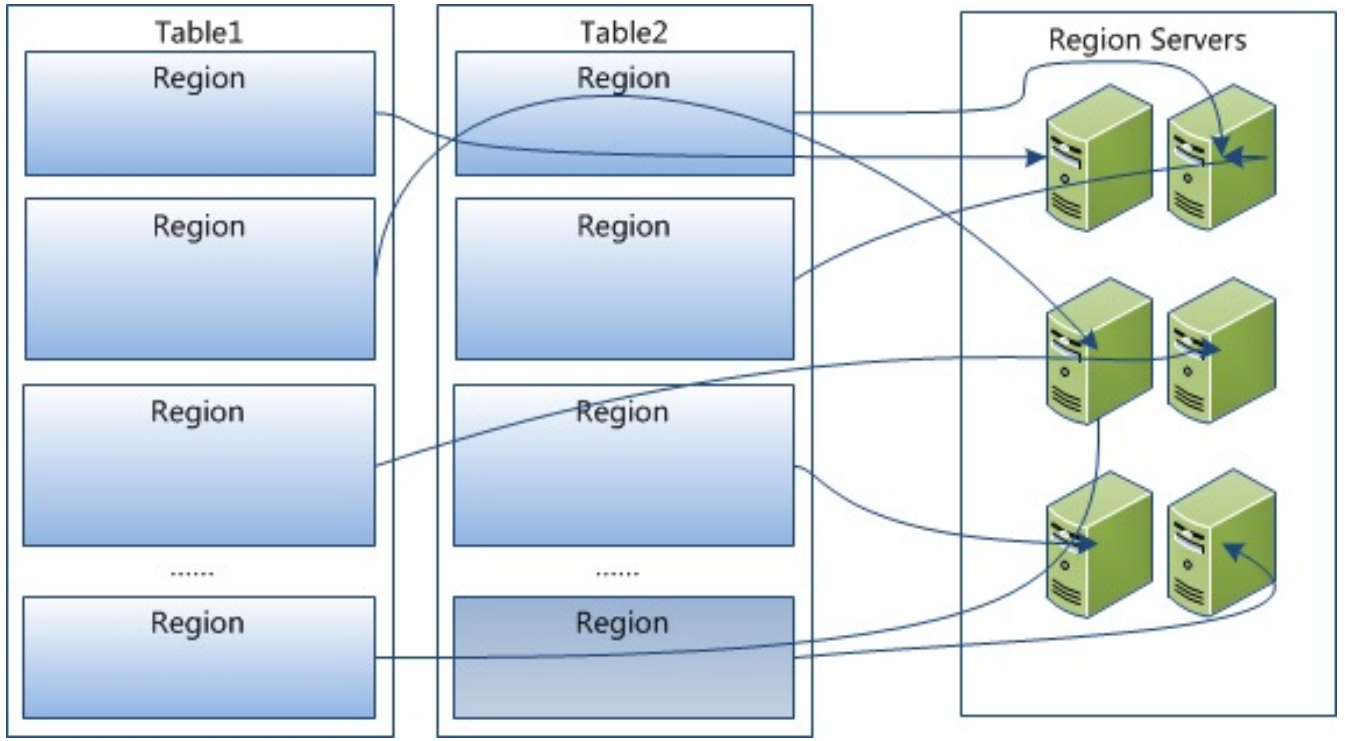
HBase的基本构架

- 由一个MasterServer和由一组子表数据区服务器RegionServer构成，分别存储逻辑大表中的部分数据。大表中的底层数据存于HDFS中。



HBase数据存储管理方法

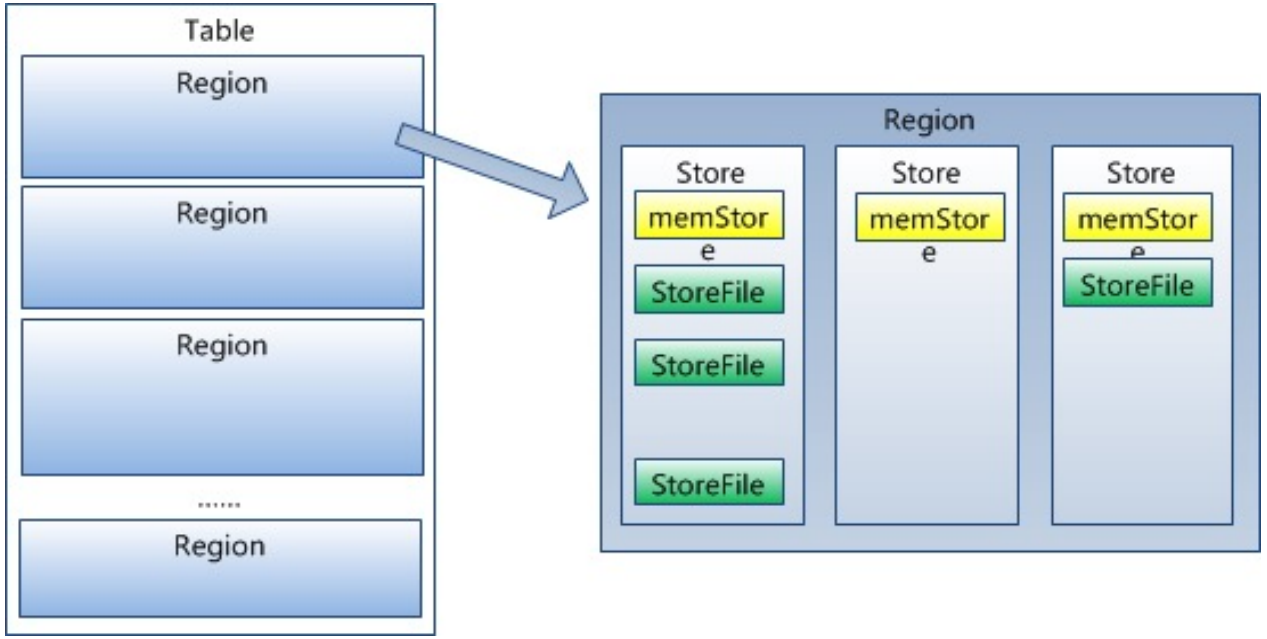
- HBase子表数据存储与子表服务器
  - 与BigTable类似，大表被分为很多个子表（Region），每个子表存储在一个子表服务器RegionServer上



- 每个Region由以下信息标识：
<表名，该Region的起始row key，创建时间>
- 每个Region的最佳大小取决于单台服务器的有效处理能力
- 同一个Region不会被分拆到多个Region服务器
- 每个Region服务器存储10-1000个Region

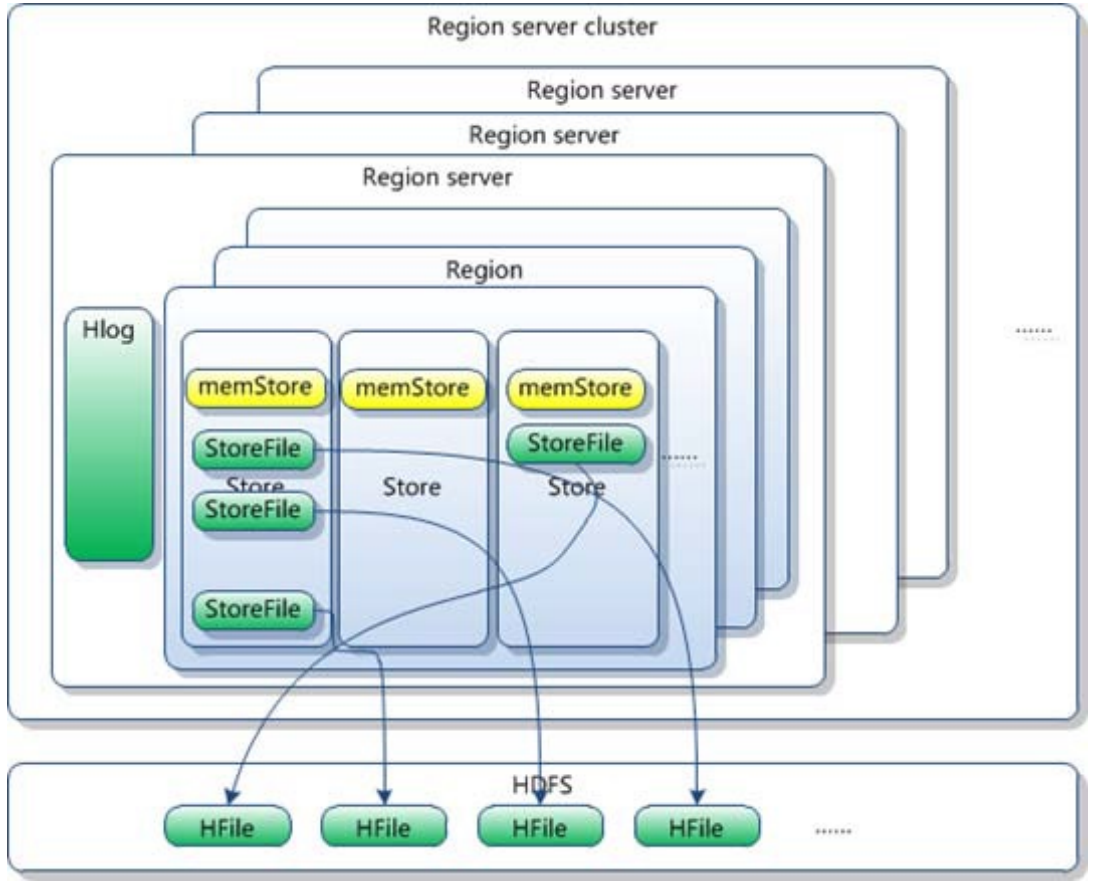
HBase数据存储管理方法

- HBase子表数据存储与子表服务器
  - 每个子表中的数据区Region由很多个数据存储块Store构成，而每个Store数据块又由存放在内存中的memStore和存放在文件中的StoreFile构成



HBase数据存储管理方法

- HBase子表数据存储与子表服务器



HBase数据存储管理方法

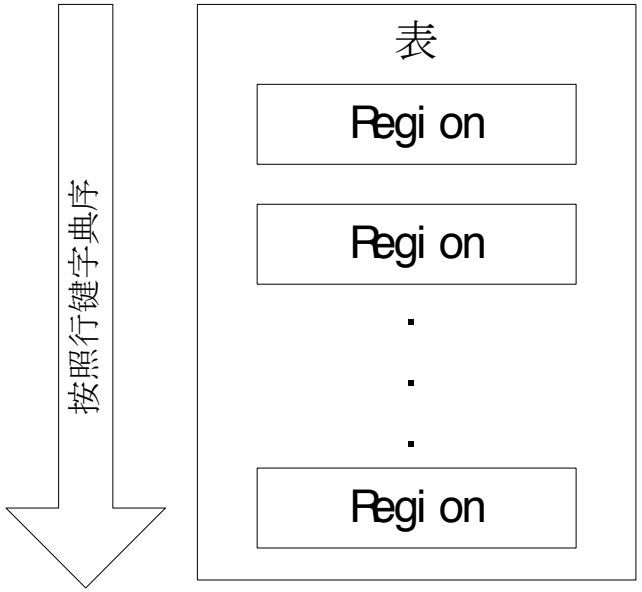
- HBase数据的访问
  - 当客户端需要进行数据更新时，先查到子表服务器，然后向子表提交数据更新请求。提交的数据并不直接存储到磁盘上的数据文件中，而是添加到一个基于内存的子表数据对象memStore中，当memStore中的数据达到一定大小时，系统将自动将数据写入到文件数据块StoreFile中。
  - 每个文件数据块StoreFile最后都写入到底层基于HDFS的文件中。



HBase数据存储管理方法

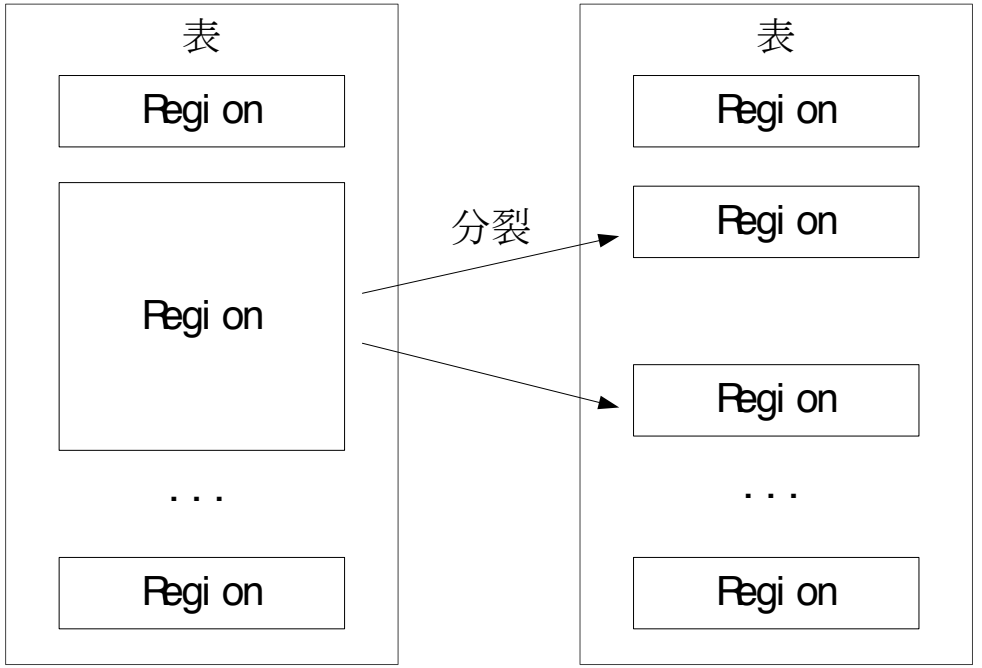
- HBase数据的访问
  - 需要查询数据时，子表先查memStore。如果没有，则再查磁盘上的StoreFile。每个StoreFile都有类似B树的结构，允许进行快速的数据查询。StoreFile将定时压缩，多个压缩为一个。
  - 两个小的子表可以进行合并；子表大到超过某个指定值时，子表服务器就需要调用HRegion.closeAndSplit()，把它分割为两个新的子表。

HBase数据存储管理方法



一个HBase表被划分成多个Region

- 开始只有一个Region，后来不断分裂
- Region拆分操作非常快，接近瞬间，因为拆分之后的Region读取的仍然是原存储文件，直到“合并”过程把存储文件异步地写到独立的文件之后，才会读取新文件



一个Region会分裂成多个新的Region



HBase数据存储管理方法

- HBase数据记录的查询定位
  - 描述所有子表和子表中数据块的元数据都存放在专门的元数据表中，并存储在特殊的子表中。子表元数据会不断增长，因此会使用多个子表来保存。而所有元数据子表的元数据都保存在根子表中。主服务器会扫描根子表，从而得到所有的元数据子表位置，再进一步扫描这些元数据子表即可获得所寻找子表的位置。



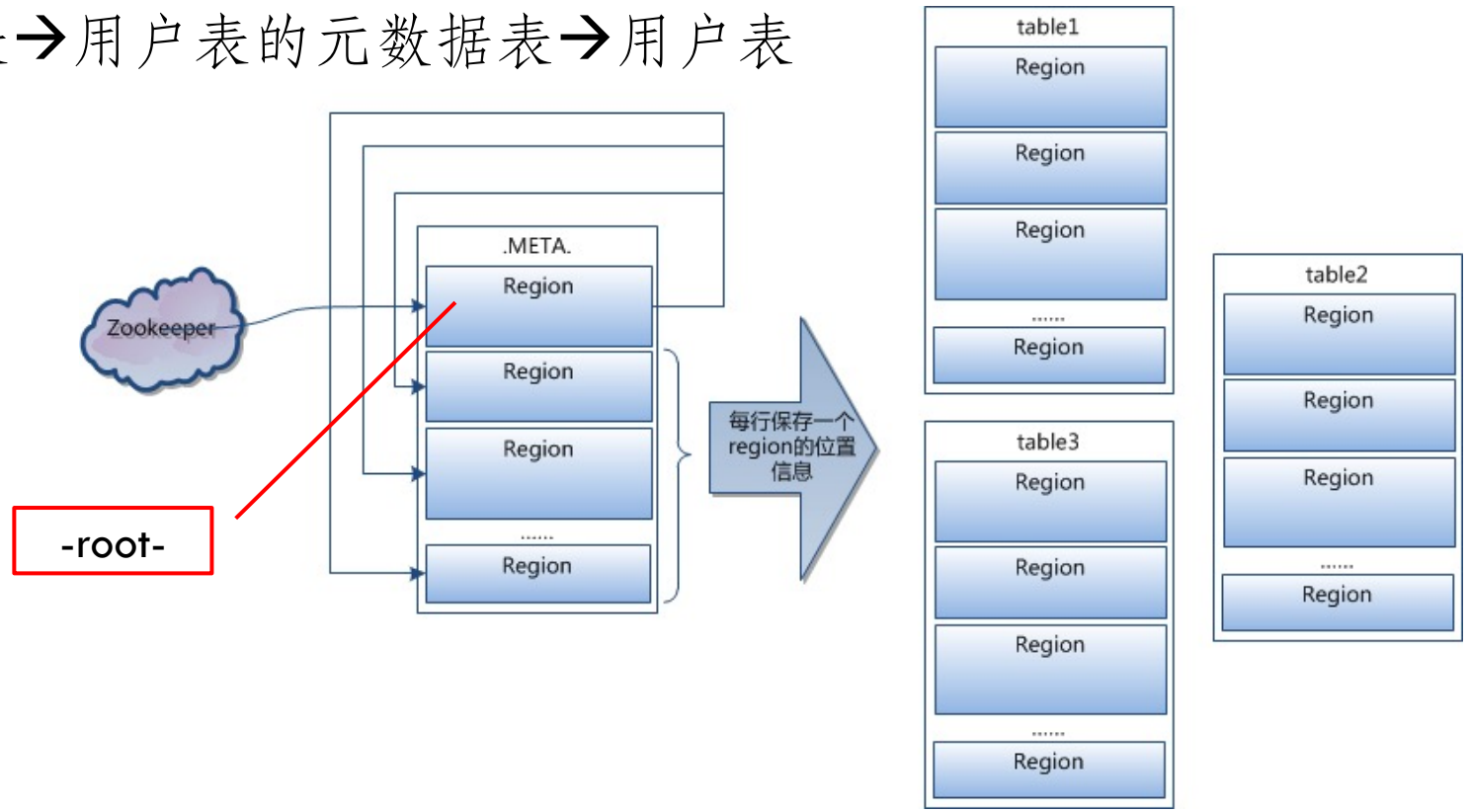
HBase数据存储管理方法

- HBase使用三层类似B+树的结构来保存Region位置
  - 通过Zookeeper里的文件得到-ROOT-表的位置，-ROOT-表永远不会被分割为多个Region
  - 通过-ROOT-表查找.META.表中相应Region的位置，为了加快访问，.META.表的全部Region的数据都会全部保存在内存中
  - 通过.META.表找到所要的用户表相应Region的位置

.META.表中，每行的row key为：<用户表名，region的起始row key，创建时间>
-ROOT-表中，每行的row key为：<.META., <用户表名，region的起始row key，创建时间>，创建时间>

HBase数据存储管理方法

- HBase数据记录的查询定位
  - 元数据子表采用三级索引结构
    - 根子表→用户表的元数据表→用户表





HBase数据存储管理方法

HBase的三层结构中各层次的名称和作用

层次	名称	作用
第一层	Zookeeper文件	记录了-ROOT-表的位置信息
第二层	-ROOT-表	记录了.META.表的Region位置信息 -ROOT-表只能有一个Region。通过-ROOT-表，就可以访问.META.表中的数据
第三层	.META.表	记录了用户数据表的Region位置信息， .META.表可以有多个Region，保存了HBase中所有用户数据表的Region位置信息

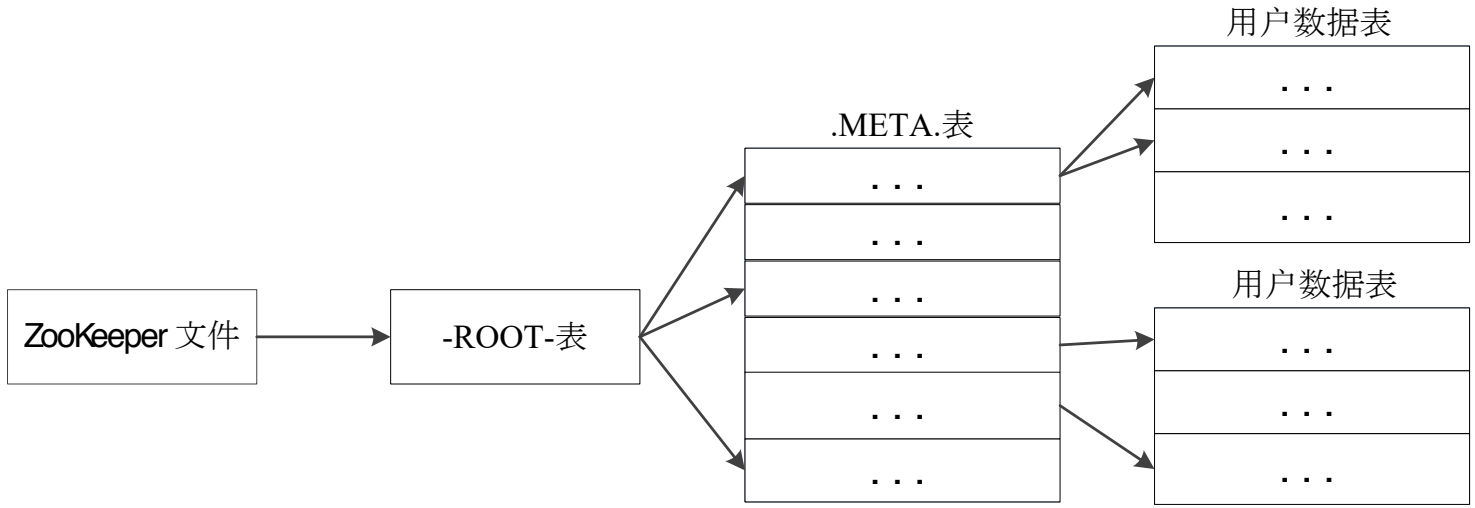


HBase数据存储管理方法

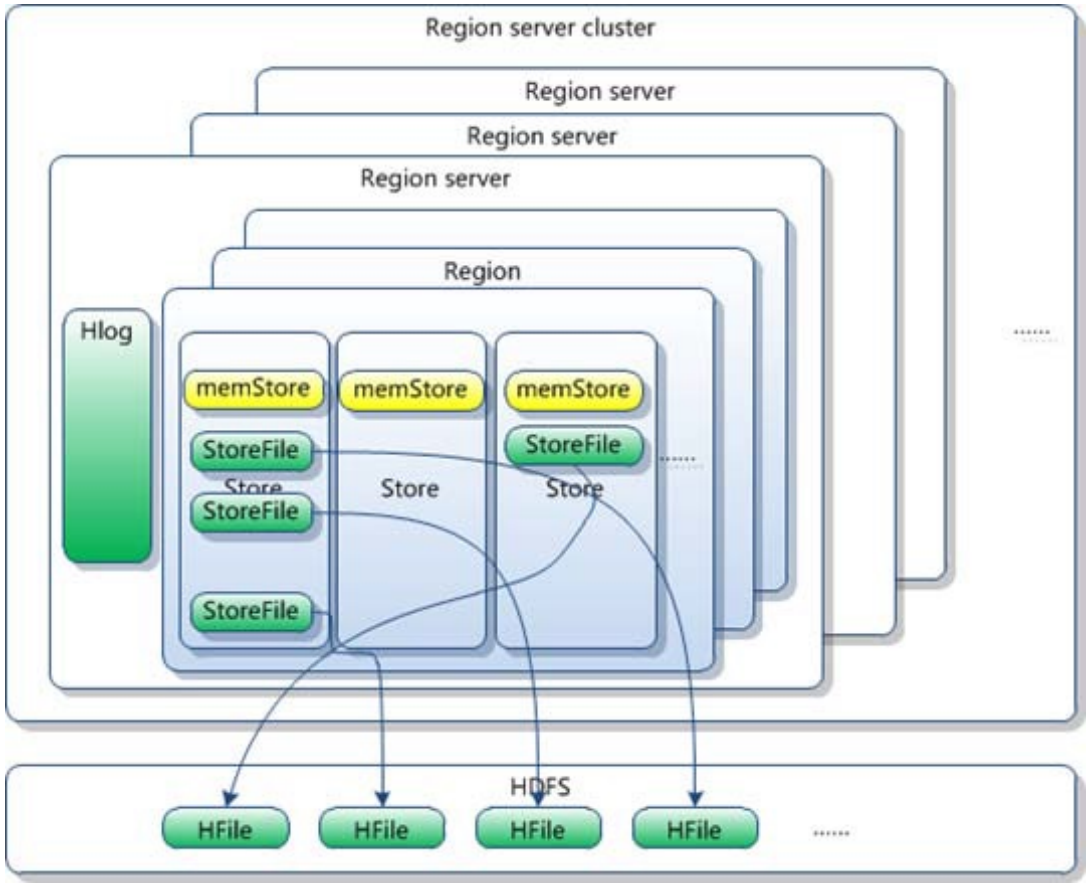
- 为了加快访问速度，.META.表的全部Region都会被保存在内存中
- 假设.META.表的每行（一个映射条目）在内存中大约占用1KB，并且每个Region限制为128MB，那么，上面的三层结构可以保存的用户数据表的Region数目的计算方法是：
- （-ROOT-表能够寻址的.META.表的Region个数）×（每个.META.表的 Region可以寻址的用户数据表的Region个数）
- 一个-ROOT-表最多只能有一个Region，也就是最多只能有128MB，按照每行（一个映射条目）占用1KB内存计算，128MB空间可以容纳128MB/1KB=2^{17}行，也就是说，一个-ROOT-表可以寻址2^{17}个.META.表的Region。
- 同理，每个.META.表的 Region可以寻址的用户数据表的Region个数是128MB/1KB=2^{17}。
- 最终，三层结构可以保存的Region数目是(128MB/1KB) × (128MB/1KB) = 2^{34}个Region

HBase数据存储管理方法

- 客户端访问数据时的“三级寻址”
  - 为了加速寻址，客户端会缓存位置信息，同时，需要解决缓存失效问题
  - 寻址过程客户端只需要询问Zookeeper服务器，不需要连接Master服务器



Region服务器工作原理



- 1. 用户读写数据过程
- 2. 缓存的刷新
- 3. StoreFile的合并



Region服务器工作原理

- 1. 用户读写数据过程
  - 用户写入数据时，被分配到相应Region服务器去执行
  - 用户数据首先被写入到MemStore和Hlog中
  - 只有当操作写入Hlog之后，commit()调用才会将其返回给客户端
  - 当用户读取数据时，Region服务器会首先访问MemStore缓存，如果找不到，再去磁盘上面的StoreFile中寻找



Region服务器工作原理

- 2. 缓存的刷新
  - 系统会周期性地把MemStore缓存里的内容刷写到磁盘的StoreFile文件中，清空缓存，并在Hlog里面写入一个标记
  - 每次刷写都生成一个新的StoreFile文件，因此，每个Store包含多个StoreFile文件
  - 每个Region服务器都有一个自己的HLog 文件，每次启动都检查该文件，确认最近一次执行缓存刷新操作之后是否发生新的写入操作；如果发现更新，则先写入MemStore，再刷写到StoreFile，最后删除旧的Hlog文件，开始为用户提供服务

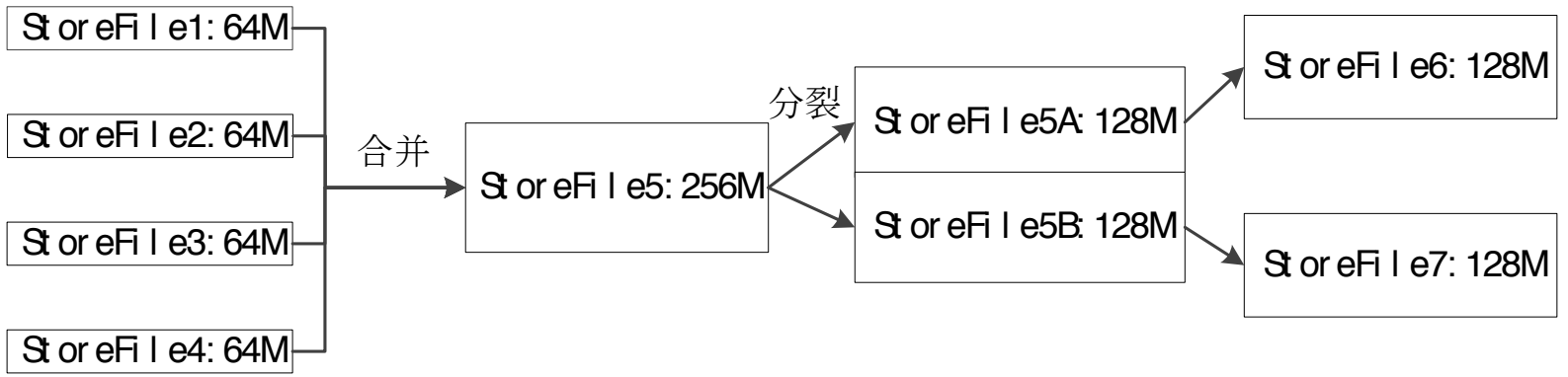


Region服务器工作原理

- 3. StoreFile的合并
  - 每次刷写都生成一个新的StoreFile，数量太多，影响查找速度
  - 调用Store.compact()把多个合并成一个
  - 合并操作比较耗费资源，只有数量达到一个阈值才启动合并

Store工作原理

- Store是Region服务器的核心
  - 多个StoreFile合并成一个
  - 单个StoreFile过大时，又触发分裂操作，1个父Region被分裂成两个子Region



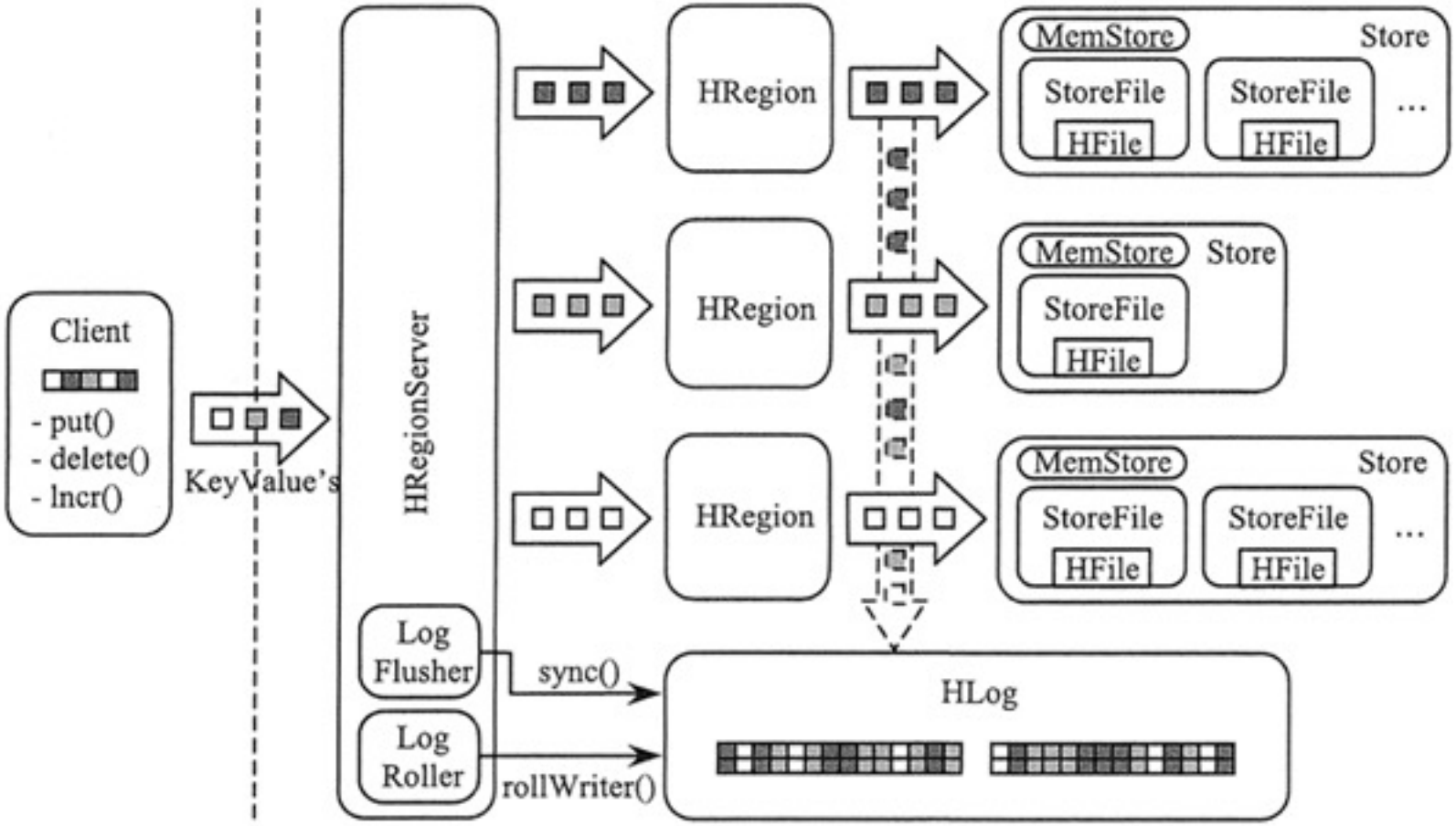
StoreFile的合并和分裂过程



HLog工作原理

- 分布式环境必须要考虑系统出错。HBase采用HLog保证系统恢复
- HBase系统为每个Region服务器配置了一个HLog文件，它是一种预写式日志（Write Ahead Log）
- 用户更新数据必须首先写入日志后，才能写入MemStore缓存，并且，直到MemStore缓存内容对应的日志已经写入磁盘，该缓存内容才能被刷写到磁盘

HLog工作原理


Client
- put()
- delete()
- Incr()
KeyValue's
HRegionServer
HRegion
HRegion
HRegion
MemStore
Store
StoreFile
HFile
StoreFile
HFile
...
MemStore
Store
StoreFile
HFile
MemStore
Store
StoreFile
HFile
StoreFile
HFile
...
Log Flusher
sync()
Log Roller
rollWriter()
HLog



HLog工作原理

- Zookeeper会实时监测每个Region服务器的状态，当某个Region服务器发生故障时，Zookeeper会通知Master
- Master首先会处理该故障Region服务器上面遗留的HLog文件，这个遗留的HLog文件中包含了来自多个Region对象的日志记录
- 系统会根据每条日志记录所属的Region对象对HLog数据进行拆分，分别放到相应Region对象的目录下，然后，再将失效的Region重新分配到可用的Region服务器中，并把与该Region对象相关的HLog日志记录也发送给相应的Region服务器
- Region服务器领取到分配给自己的Region对象以及与之相关的HLog日志记录以后，会重新做一遍日志记录中的各种操作，把日志记录中的数据写入到MemStore缓存中，然后，刷新到磁盘的StoreFile文件中，完成数据恢复
- 共用日志优点：提高对表的写操作性能；缺点：恢复时需要分拆日志



HBase存储格式

- HBase中的所有数据文件都存储在Hadoop HDFS文件系统上，主要包括两种文件类型：
  - HFile，HBase中KeyValue数据的存储格式，HFile是Hadoop的二进制格式文件，实际上StoreFile就是对HFile做了轻量级包装，即StoreFile底层就是HFile
  - HLogFile，HBase中WAL（Write Ahead Log）的存储格式，物理上是Hadoop的Sequence File



摘要

- HBase基本工作原理
- HBase基本操作
- HBase编程方法示例



HBase

- 官网：
  - http://hbase.apache.org
- 最新版本：
  - 3.0.0-alpha-4 Jun 7, 2023
  - 2.5.6 Oct 20, 2023
  - 2.4.17 Apr 6, 2023
- 参考指南：
  - http://hbase.apache.org/book.html



HBase

- 版本兼容问题

HBase Version	JDK 6	JDK 7	JDK 8	JDK 11
HBase 2.3+	✗	✗	✓	!*
HBase 2.0-2.2	✗	✗	✓	✗
HBase 1.2+	✗	✓	✓	✗
HBase 1.0-1.1	✗	✓	!	✗
HBase 0.98	✓	✓	!	✗
HBase 0.94	✓	✓	✗	✗

	HBase-2.4.x	HBase-2.5.x
Hadoop-2.10.[0-1]	✓	✗
Hadoop-2.10.2+	✓	✓
Hadoop-3.1.0	✗	✗
Hadoop-3.1.1+	✓	✗
Hadoop-3.2.[0-2]	✓	✗
Hadoop-3.2.3+	✓	✓
Hadoop-3.3.[0-1]	✓	✗
Hadoop-3.3.2+	✓	✓



HBase配置和安装

- hbase-env.sh
  - JAVA_HOME
  - HBASE_MANAGES_ZK (默认为true，可能需要取消注释，改成false)
  - HBASE_DISABLE_HADOOP_CLASSPATH_LOOKUP (默认为false，可能需要取消注释，改成true)
- hbase-site.xml
  - Standalone模式
  - Distributed模式
    - 单机伪分布式模式/集群模式
- Web Interface：
  - http://localhost:16010



HBase配置和安装

- 前提
  - JDK
  - Hadoop（单机模式不需要，伪分布模式和集群模式需要）
  - SSH
- 启动关闭Hadoop和HBase的顺序一定是：
  启动Hadoop → 启动HBase → 关闭HBase → 关闭Hadoop
- HBASE_MANAGES_ZK=true，则由HBase自己管理Zookeeper，否则，启动独立的Zookeeper
- 建议：单机版HBase，使用自带Zookeeper；集群安装HBase则采用单独Zookeeper集群



HBase基本操作与编程方法

- HBase Shell操作
  - HBase Shell常用的操作命令有create, describe, disable, enable, drop, list, scan, put, get, delete, deleteall, count, status等，通过help可以看到详细的用法。
  - 在HBase Shell语法中，所有字符串参数都必须包含在单引号中，且区分大小写。



HBase Shell操作

- 表的管理
  - 查看有哪些表
    - list
  - 创建表
    - create <table>, {NAME => <family>, VERSIONS => <VERSIONS>}
  - 删除表
    - disable <table>
    - drop <table>
  - 查看表结构
    - describe <table>
  - 修改表结构
    - alter 't1', {NAME => 'f1'}, {NAME => 'f2', METHOD => 'delete'}



HBase Shell操作

- 表数据的增删查改
  - 添加数据
    - put <table>,<rowkey>,<family:column>,<value>,<timestamp>
  - 查询数据
    - 查询某行记录
      - get <table>,<rowkey>,[<family:column>,....]
    - 扫描表
      - scan <table>, {COLUMNS => [<family:column>,....], LIMIT => num}
    - 查询表中的数据行数
      - count <table>, {INTERVAL => intervalNum, CACHE => cacheNum}



HBase Shell操作

- 表数据的增删查改
  - 删除数据
    - 删除行中的某个列值
      - delete <table>, <rowkey>, <family:column> , <timestamp>
      - 必须指定列名
    - 删除行
      - deleteall <table>, <rowkey>, <family:column> , <timestamp>
      - 可以不指定列名，删除整行数据查询表中的数据行数
    - 删除表中的所有数据
      - truncate <table>



HBase Shell操作

- 我们建立如下的表以及数据

行键	列族 StuInfo				列族 Grades		
	Name	Age	Sex	Class	BigData	Computer	Math
0001	Tom Green	18	Male		80	90	85
0002	Amy	19		01	95		89
0003	Allen	19	Male	02	90		88



创建表

- HBase使用create命令创建表，创建表时需要指明表名和列族名
  - create 'Student', 'StuInfo', 'Grades'
- 如果建表时要设置列族的参数，比如
  - create 'Student', {NAME => 'StuInfo', VERSIONS => 3}, {NAME =>'Grades', BLOCKCACHE => true}
    - 大括号内是对列族的定义，NAME、VERSION 和 BLOCKCACHE 是参数名，无须使用单引号，符号=>表示将后面的值赋给指定参数。
- 创建表结构以后，可以使用exists命令查看是否存在，或使用list命令查看数据库中所有表，还可以使用describe命令查看指定表的列族信息。



查看表

- 创建表结构以后，可以使用exists命令查看是否存在，或使用list命令查看数据库中所有表，还可以使用describe命令查看指定表的列族信息。

```
hbase(main):005:0> exists 'Student'
Table Student does exist
0 row(s) in 0.2080 seconds

hbase(main):006:0> list
TABLE
Student
hbase_thrift
student
test
8 row(s) in 0.0560 seconds

=> ["Student", "hbase_thrift", "student", "test"]
```

```
hbase(main):006:0> describe 'Student'
Table Student is ENABLED
Student
COLUMN FAMILIES DESCRIPTION
{NAME => 'Grades', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false', KEEP_DE
LETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', COMPRESSION =>
 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_S
COPE => '0'}
{NAME => 'StuInfo', BLOOMFILTER => 'ROW', VERSIONS => '3', IN_MEMORY => 'false', KEEP_D
ELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', COMPRESSION =
> 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_
SCOPE => '0'}
2 row(s) in 0.0340 seconds
```



修改表

- 修改列族
  - alter 'Student', {NAME => 'Grades', VERSIONS => 3}
  - 注意：修改已有数据的列族属性时，HBase需要对列族里所有数据进行修改，如果数据量较大，则修改可能要占用很长时间
- 增加列族
  - alter 'Student', 'hobby'
- 删除列族
  - alter 'Student', { NAME => 'hobby', METHOD => 'delete' }
  - alter 'Student', 'delete' => 'hobby'
  - 注意： HBase 表至少要包含一个列族，因此当表中只有一个列族时，无法将其删除。



删除表

- HBase使用drop命令删除表，但是在删除表之前需要先使用disable命令禁用表。例如：
  - disable 'Student'
  - drop 'Student'
- 如果只是想清空表中的所有数据，使用 truncate 命令即可，此命令相当于完成禁用表、删除表，并按原结构重新建立表操作
  - truncate 'Student'



插入数据

- HBase 使用 put 命令向数据表中插入数据，put 向表中增加一个新行数据，或覆盖指定行的数据。例如：
  - put 'Student', '0001', 'StuInfo:Name', 'Tom Green', 1
    - ‘Student’为表名；‘0001‘为行键的名称，为字符串类型； ’StuInfo:Name‘为列族和列的名称，中间用冒号隔开，列族名必须是已经创建的，否则HBase会报错，列名是临时定义的，因此列族里的列可以随意扩展； ’Tom Green’为单元格的值，所有数据都是字符串的形式；’1’为时间戳，如果不设置时间戳，则系统会自动插入当前时间为时间戳。
    - 如果 put 语句中的单元格是已经存在的，即行键、列族及列名都已经存在，且不考虑时间戳的情况下，执行 put 语句，则可对数据进行更新操作。例如：
      - put 'Student', '0001', 'StuInfo:Name', 'Jim Green‘
    - 如果在初始创建表时，已经设定了列族VERSIONS参数值为n，则put可以保存n个版本的数据。



删除数据

- HBase 使用 delete 命令可以从表中删除一个单元格或一个行集，语法和put类似，必须知名表名和列族名称，而列名和时间戳是可选的。例如：
  - delete 'Student', '0002', 'Grades'
    - 需要注意的是，delete 操作并不会马上删除数据，只会将对应的数据打上删除标记（tombstone），只有在合并数据时，数据才会被删除。
  - delete命令不能跨列族操作，如需删除表中所有列族在某一行上的数据，即删除上表中一个逻辑行，则需要使用 deleteall 命令，如下所示，不需要指定列族和列的名称：
    - deleteall 'Student', '0001'

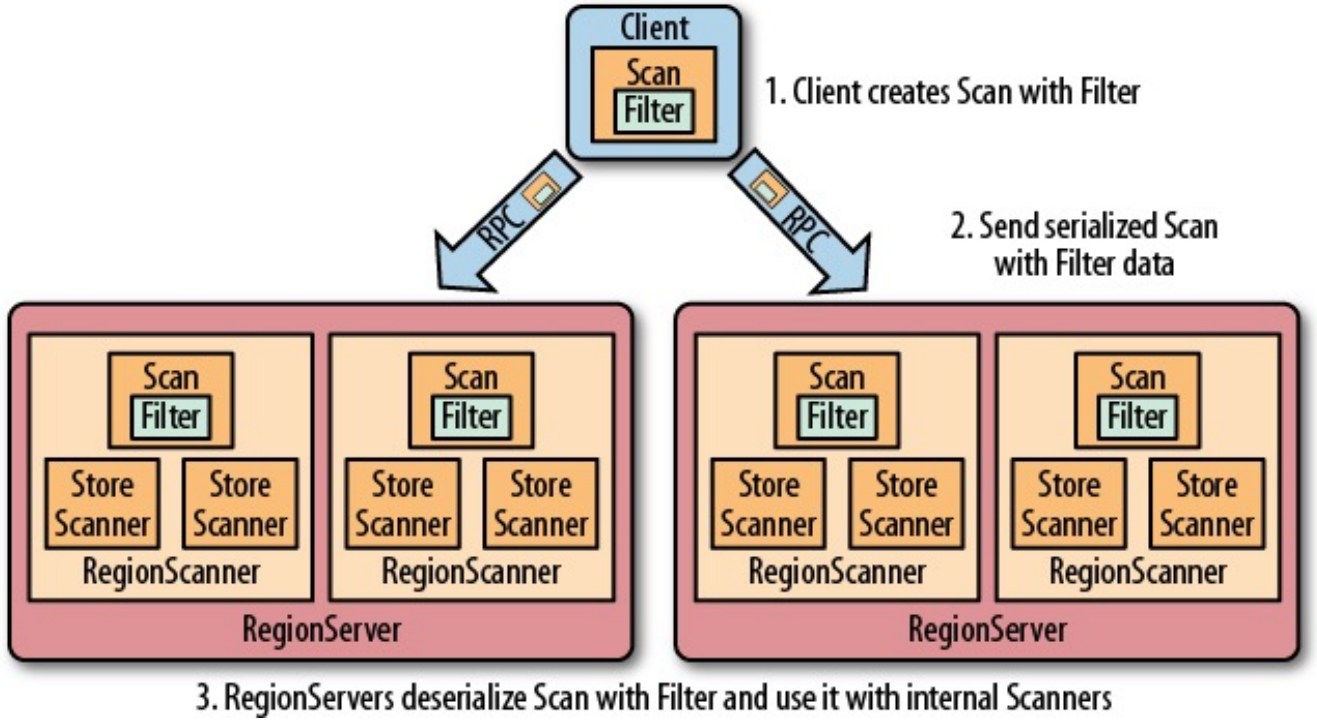


获取数据和查询全表数据

- HBase 使用 get 命令可以从数据表中获取某一行记录，类似于关系型数据库中的 select 操作。get 命令必须设置表名和行键名，同时可以选择指明列族名称、时间戳范围、数据版本等参数。例如：
  - get 'Student', '0001'
- HBase 使用 scan 命令用来查询全表数据，使用时只需要指定表名即可。例如：
  - scan 'Student'
  - scan只从条件输出时，需要使用大括号将参数包含起来，例如：
    - scan 'Student', {COLUMNS =>'Grades'}

HBase的过滤器

- HBase 提供了种类丰富的过滤器（filter）来提高数据处理的效率，用户可以通过内置或自定义的过滤器来对数据进行过滤，所有的过滤器都在服务端生效，即谓词下推（predicate push down）。这样可以保证过滤掉的数据不会被传送到客户端，从而减轻网络传输和客户端处理的压力。





HBase的过滤器

- 在HBase中，get和scan操作都可以使用过滤器来设置输出的范围，类似SQL里的where查询条件。
- 使用show_filters命令可以查看当前HBase支持的过滤器类型。

```
hbase(main):009:0> show_filters
ColumnPrefixFilter
TimestampsFilter
PageFilter
MultipleColumnPrefixFilter
FamilyFilter
ColumnPaginationFilter
SingleColumnValueFilter
RowFilter
QualifierFilter
ColumnRangeFilter
ValueFilter
PrefixFilter
SingleColumnValueExcludeFilter
ColumnCountGetFilter
InclusiveStopFilter
DependentColumnFilter
FirstKeyOnlyFilter
KeyOnlyFilter
```



HBase的过滤器

- 在使用上述过滤器时，一般需要配合比较运算符或比较器使用。

比较运算符

比较运算符	描述
=	等于
>	大于
>=	大于等于
<	小于
<=	小于等于
!=	不等于

比较器

比较器	描述
BinaryComparator	匹配完整字节数组
BinaryPrefixComparator	匹配字节数组前缀
BitComparator	匹配比特位
NullComparator	匹配空值
RegexStringComparator	匹配正则表达式
SubstringComparator	匹配子字符串

- 使用过滤器的语法格式：
  - scan '表名', {Filter =>"过滤器(比较运算符, '比较器')" }
  - Filter=> 指明过滤的方法，整体可用大括号引用，也可以不用大括号。过滤的方法用双引号引用，而比较方式用小括号引用。

HBase的常用过滤器

- 行键过滤器RowFilter
  - 可以配合比较器和运算符，实现行键字符串的比较和过滤。例如，匹配行键中大于 0001 的数据，可使用 binary 比较器；匹配以 0001 开头的行键，可使用 substring 比较器，注意 substring 不支持大于或小于运算符。

```
hbase(main):002:0> scan 'Student', FILTER=>"RowFilter(=,'substring:0001')"
ROW                  COLUMN+CELL
 0001                column=Grades:BigData, timestamp=1541039459116, value=80
 0001                column=Grades:Computer, timestamp=1541039459299, value=90
 0001                column=Grades:Math, timestamp=1541039459430, value=85
 0001                column=StuInfo:Age, timestamp=1541039335956, value=18
 0001                column=StuInfo:Name, timestamp=4, value=curry
 0001                column=StuInfo:Sex, timestamp=1541039336280, value=Male
1 row(s) in 0.1750 seconds

hbase(main):003:0> scan 'Student', FILTER=>"RowFilter(>,'binary:0001')"
ROW                  COLUMN+CELL
 0002                column=Grades:Math, timestamp=1541040264452, value=85
1 row(s) in 0.0940 seconds
```



HBase的过滤器

- 针对行键进行匹配的过滤器还有 PrefixFilter、KeyOnlyFilter、FirstKeyOnlyFilter 和 InclusiveStopFilter。

行键过滤器	描述	示例
PrefixFilter	行键前缀比较器，比较行键前缀	scan 'Student', FILTER => "PrefixFilter('0001')" scan 'Student', FILTER => "RowFilter(=,'substring:0001')"
KeyOnlyFilter	只对单元格的键进行过滤和显示，不显示值	scan 'Student', FILTER => "KeyOnlyFilter()"
FirstKeyOnlyFilter	只扫描显示相同键的第一个单元格，其键值对会显示出来	scan 'Student', FILTER => "FirstKeyOnlyFilter()"
InclusiveStopFilter	替代 ENDROW 返回终止条件行	scan 'Student', { STARTROW => '0001', FIILTER => "InclusiveStopFilter('binary:0002')" } scan 'Student', { STARTROW => '0001', ENDROW => '0003' }



HBase的过滤器

- 列族与列过滤器
  - 针对列族进行过滤的过滤器为 FamilyFliter，其语法结构与 RowFilter 类似，不同之处在于 FamilyFilter 是对列族名称进行过滤的。例如：
    - scan 'Student', FILTER=>" FamilyFilter(= , 'substring:Grades')"
  - 针对列的过滤器如下，这些过滤器也需要结合比较运算符和比较器进行列族或列的扫描过滤。

列过滤器	描述	示例
QualifierFilter	列标识过滤器，只显示对应列名的数据	scan 'Student', FILTER => "QualifierFilter(=,'substring:Math')"
ColumnPrefixFilter	对列名称的前缀进行过滤	scan 'Student', FILTER => "ColumnPrefixFilter('Ma')"
MultipleColumnPrefixFilter	可以指定多个前缀对列名称过滤	scan 'Student', FILTER => "MultipleColumnPrefixFilter('Ma','Ag')"
ColumnRangeFilter	过滤列名称的范围	scan 'Student', FILTER => "ColumnRangeFilter('Big',true,'Math',false')"



HBase的过滤器

- 值过滤器：针对单元格进行扫描的过滤器
  - ValueFilter 过滤器可以利用 get 和 scan 方法对单元格进行过滤，但是使用 get 方法时，需要指定行键。SingleColumnValueFilter 和 SingleColumnValueExcludeFilter 过滤器扫描的结果是相反的， 都需要在过滤条件中指定列族和列的名称。

值过滤器	描述	示例
ValueFilter	值过滤器，找到符合值条件的键值对	scan 'Student', FILTER => "ValueFilter(=,'substring:curry')" get 'Student', '0001', FILTER => "ValueFilter(=,'substring:curry')"
SingleColumnValueFilter	在指定的列族和列中进行比较的值过滤器	scan 'Student', Filter => "SingleColumnValueFilter('StuInfo', 'Name', =, 'binary:curry')"
SingleColumnValueExcludeFilter	S排除匹配成功的值	scan 'Student', Filter => "SingleColumnValueExcludeFilter('StuInfo', 'Name', =, 'binary:curry')"



HBase的过滤器

- 其他过滤器

值过滤器	描述	示例
ColumnCountGetFilter	限制每个逻辑行返回键值对的个数，在 get 方法中使用	get 'Student', '0001', FILTER => "ColumnCountGetFilter(3)"
TimestampsFilter	时间戳过滤，支持等值，可以设置多个时间戳	scan 'Student', Filter => "TimestampsFilter(1,4)"
InclusiveStopFilter	设置停止行	scan 'Student', { STARTROW => '0001', ENDROW => '0005', FILTER => "InclusiveStopFilter('0003')" }
PageFilter	对显示结果按行进行分页显示	scan 'Student', { STARTROW => '0001', ENDROW => '0005', FILTER => "PageFilter(3)" }
ColumnPaginationFilter	对一行的所有列分页，只返回[offset,offset+limit] 范围内的列	scan 'Student', { STARTROW => '0001', ENDROW => '0005', FILTER => "ColumnPaginationFilter(2,1)" }



HBase中的disable和enable

- disable和enable都是HBase中比较常见的操作，很多对table的修改都需要表在disable的状态下才能进行
- disable 'students'将表students的状态更改为disable的时候，HBase会在zookeeper中的table结点下做记录
- 在zookeeper记录下该表的同时，还会将表的region全部下线，region为offline状态
- enable的过程和disable相反，会把表的所有region上线，并删除zookeeper下的标志。如果在enable前，META中有region的server信息，那么此时会在该server上将该region 上线；如果没有server的信息，那么此时还要随机选择一台机器作为该region的server



摘要

- HBase基本工作原理
- HBase基本操作
- HBase编程方法示例



HBase的API接口

- HBase提供了丰富的API接口让用户去操作这些数据。主要的API接口有3个，Put，Get，Scan。Put和Get是操作指定行的数据的，所以需要提供行键来进行操作。Scan是操作一定范围内的数据，通过指定开始行键和结束行键来获取范围，如果没有指定开始行键和结束行键，则默认获取所有行数据。



HBase的Java编程

- HBase Java编程接口概述HBaseConfiguration
  - HBaseConfiguration是每一个HBase client都会使用到的对象，它代表的是HBase配置信息。
  - 默认的构造方式会尝试从hbase-default.xml和hbase-site.xml中读取配置。如果classpath没有这两个文件，就需要你自己设置配置。
  - 配置Java代码示例：

```
Configuration HBASE_CONFIG = new Configuration();
HBASE_CONFIG.set("hbase.zookeeper.quorum", "zkServer");
HBASE_CONFIG.set("hbase.zookeeper.property.clientPort", "2181");
HBaseConfiguration cfg= new BaseConfiguration(HBASE_CONFIG);
```



HBase的Java编程：创建表

- 创建表是通过Admin对象来操作的。Admin负责表的META信息处理。Admin提供了createTable这个方法：
  - public void createTable(HTableDescriptor desc)
- HTableDescriptor代表的是表schema
- HColumnDescriptor代表的是column的schema



HBase的Java编程：创建表

- Java代码示例：

```
Connection conn= ConnectionFactory.createConnection(config);
Admin  admin= conn.getAdmin();
HTableDescriptor  t = new HTableDescriptor(tableName);
t.addFamily(new HColumnDescriptor("f1"));
t.addFamily(new HColumnDescriptor("f2"));
t.addFamily(new HColumnDescriptor("f3"));
t.addFamily(new HColumnDescriptor("f4"));
admin.createTable(t);
```



HBase的Java编程：插入数据

- Table通过put方法来插入数据，可以传递单个批Put对象或者List put对象来分别实现单条插入和批量插入。Put对象的常用方法：

```
public static void addData(String tableName, String rowKey, String family,  String qualifier, String value) throws Exception{
   try{
      Connection conn= ConnectionFactory.createConnection(config);
      Table table =conn.getTable(tableName);
      Put put= new Put(Bytes.toBytes(rowKey));
      put.add(Bytes.toBytes(family), Bytes.toBytes(qualifier),Bytes.toBytes(value));
      table.put(put);
      System.out.println("insert record success!");
    } catch(IOException e) { e.printStackTrace(); }
}
```



HBase的Java编程：删除表

- 删除表也通过Admin来操作，删除表之前首先要disable表。这是一个非常耗时的操作，所以不建议频繁删除表。disableTable和deleteTable分别用来disable和delete表
- Java代码示例：

```
if(admin.tableExists(tableName)){
        admin.disableTable(tableName);
        admin.deleteTable(tableName);
}
```



HBase的Java编程：查询数据

- 查询分为单条随机查询和批量查询
  - 单条查询是通过row key在table中查询某一行的数据。Table提供了get方法来完成单条查询。
  - 批量查询是通过制定一段row key的范围来查询。Table提供了个getScanner方法来完成批量查询。
- Java代码示例：

```
Scan s = new Scan();   s.setMaxVersions();
ResultScanner  ss= table.getScanner(s);
for(Result r:ss){
   System.out.println(new String(r.getRow()));
   for(KeyValuekv:r.raw()){  System.out.println(new String(kv.getColumn()));}
}
```



HBase的Java编程：删除数据

- Table通过delete方法来删除数据：delete(final Delete delete)
- Delete常用方法：
  - deleteFamily或deleteColumns：
  - 指定要删除的family或者column的数据。如果不调用任何这样的方法，将会删除整行

```
Table table = conn.getTable("mytest");
Delete d = new Delete("row1".getBytes());
table.delete(d)
```



HBase的Java编程：切分表

- 参数hbase.hregion.max.filesize指示在当前ReigonServer上单个Reigon的最大存储空间，单个Region超过该值时，这个Region会被自动split成更小的region。
  - Admin提供split方法来将table进行手工split。
    - public void split(TableName tableName)
  - Admin提供splitRegion方法来将region进行手工split。
    - public void splitRegion(byte[] regionName)
  - 由于split是一个异步操作，并不能确切地控制region的个数。



HBase的Java编程：程序的编写与运行

- 官网教程：
  - https://hbase.apache.org/book.html#_examples
- 与Hadoop编程类似，在进行HBase Java编程的时候也需要配置相关的类包文件jar文件，在程序执行的时候，还需要给出类库的目录。
  - 在开发环境中准备导入jar包：
  - 将HBase目录中的jar包以及lib中的所有jar包都加入
- 或者使用maven构建，参考代码：
  - https://github.com/yuping-nju/bdkit-demo/blob/master/hbase-java-demo

THANK YOU



南京大學
NANJING UNIVERSITY

南京大学计算机软件研究所
Institute of Computer Software, Nanjing University